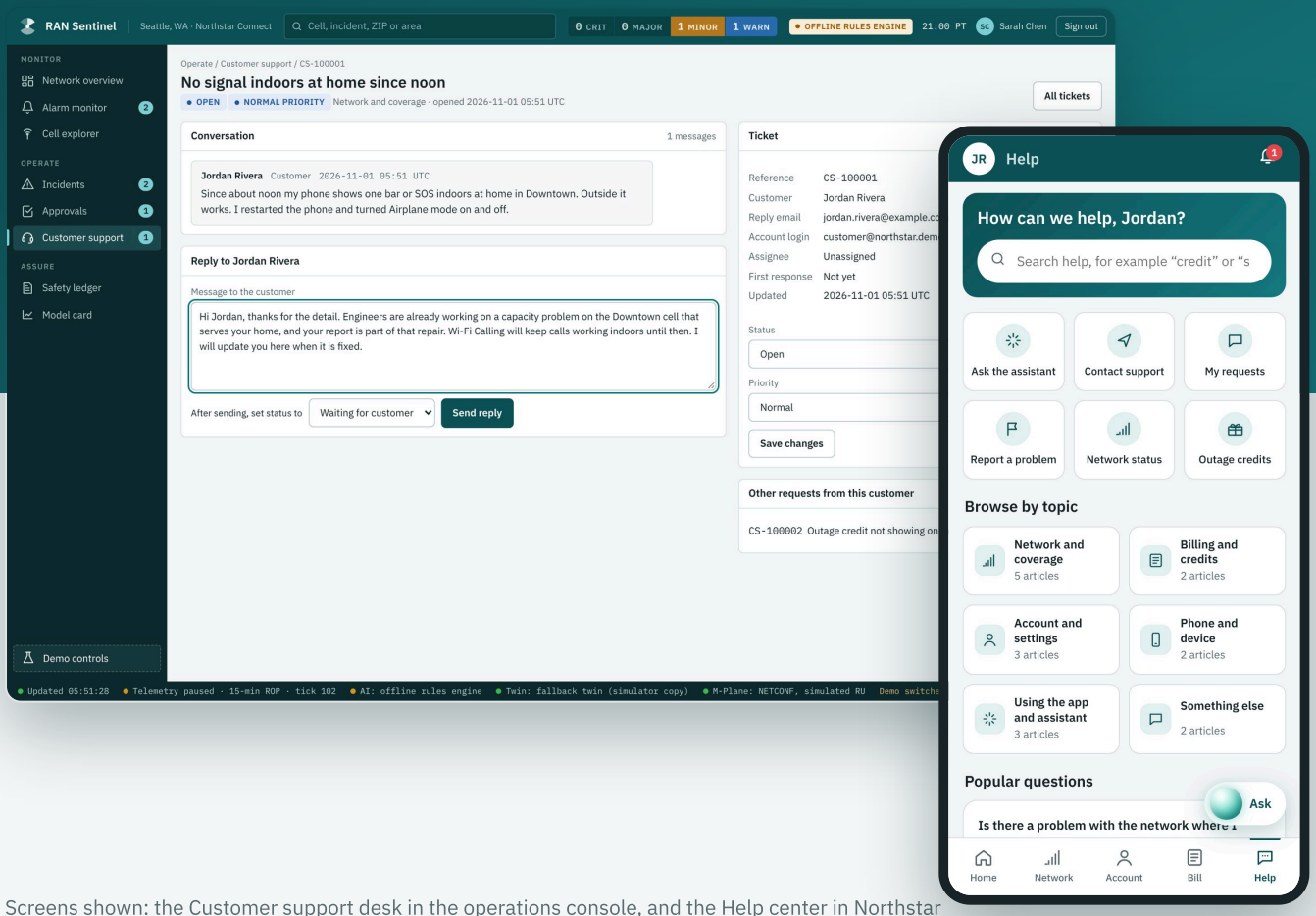
RAN Sentinel

COMPLETE USER MANUAL · VERSION 1.0

Install it, use it, test it, show it.

The operations console, the Northstar Connect customer app and the new Help & Support module, from a fresh clone to a judged demonstration.

Installation · User guide · Annotated screenshots · Testing guide · Judges' demo · Troubleshooting



Screens shown: the Customer support desk in the operations console, and the Help center in Northstar Connect, captured from the running application. Everything runs on a simulated network for a fictional operator.

EDITION
1 November 2026

SOFTWARE
branch main, commit ebfbfda

AUTHOR
AKMaasif (QuanTell)

LICENCE
MIT

DOCUMENT INFORMATION

About this manual

This manual explains how to install, run, use, test and demonstrate RAN Sentinel, including the new Customer Care and Help & Support module. It is written for three readers: someone cloning the repository for the first time, an engineer or care agent using the two applications, and a judge who wants to see the important parts in a few minutes.

Title	RAN Sentinel: Complete User Manual
Version	1.0, submission edition
Date	1 November 2026
Software covered	RAN Sentinel, branch `main`, including the Help & Support module (commit ebfbfda)
Author	AKMaasif (QuanTell)
Repository	github.com/AKMaasif/Meta-Ai-Global-Hackathon-Ran-Sentinel-
Licence	MIT
Companion document	`docs/RAN_Sentinel_Technical_Documentation.pdf` (industry case and technical manual)

How to read it

Sections A and B explain what the system is and how it is built. Section C takes you from an empty folder to a running application, and every command is labelled with the folder it must be run in. Section D is the user guide, one feature at a time. Section E is a visual walkthrough: every screenshot in it was taken from the running application on 1 November 2026, and the numbered markers on each picture match the numbered notes below it. Section F describes the automated tests and lists the results that were actually produced. Section G is a timed demonstration script for judges, and Section H collects known problems and their fixes.

Commands appear in grey boxes. Text in `monospace` is something you type or a file you open. Where a step differs between Windows and macOS or Linux, both are shown.

> **What is real and what is simulated.** RAN Sentinel runs on a simulated network of 36 LTE cells in Seattle and a fictional operator, Northstar Connect, with one fictional subscriber, Jordan Rivera. No real network, account or payment is involved. The support desk is staffed by the demo engineer account, Sarah Chen. Nothing in this manual claims a result that was not run; where something was not tested it says so.

Revision history

Version	Date	Change
1.0	1 Nov 2026	First complete manual. Adds the Help center, contact form, support requests and the staff support desk, with test results.

CONTENTS

Contents

A. Introduction 5
A1 What RAN Sentinel is 5
A2 Who uses it 5
A3 What is real and what is simulated 5
A4 Feature overview 5

B. Architecture 6
B1 System overview 6
B2 Components 7
B3 Database 7
B4 API flow for a support request 8
B5 Authentication and access control 9

C. Installation and setup 9
C1 Requirements 9
C2 Get the code 9
C3 Windows: one-click start 10
C4 Step by step 10
C5 Configuration 10
C6 Database setup 11
C7 First run and sign-in 11
C8 Optional components 11
C9 Check the installation 11

D. User guide 11
D1 Signing in to the operations console 11
D2 Monitoring the network 12
D3 Following an incident 12
D4 Approvals, field evidence and sign-off 12
D5 Customer support desk (staff) 13
D6 Safety ledger, model card and demo controls 13
D7 Customer app: sign-in, home, network and reports 13
D8 Account, bill and notifications 14
D9 The assistant and the voice agent 14
D10 Help center and FAQs 14
D11 Contacting support and following a request 14

E. Annotated screenshots 16
E1 Operations console 16
E2 Northstar Connect 22
E3 Help & Support 23
E4 Other screen sizes 25

F. Testing guide 26
F1 How to run the tests 26

F2 Results of this run 26
F3 Support module test cases (executed) 27
F4 Browser checks (executed) 28
F5 Recommended tests not yet executed 28

G. Project demonstration guide for judges 29
G1 Before you start (2 minutes, off the clock) 29
G2 The eight-minute route 29
G3 If something goes wrong during the demo 30

H. Troubleshooting and FAQ 30
H1 Common problems 30
H2 Frequently asked questions 31

Appendix 31
X1 Folder structure 31
X2 Support data limits 32
X3 Glossary 32
X4 Related documents 32

List of figures

Figure 1. Operations console sign-in. 16
Figure 2. Network overview after two demo faults were created. 17
Figure 3. Alarm monitor. 17
Figure 4. Cell explorer for SEA-RAN-001. 18
Figure 5. Incident list. 18
Figure 6. Incident workspace for a capacity problem awaiting the RAN Optimisation decision. 19
Figure 7. Approvals: one card per team and the decisions waiting for each. 19
Figure 8. Field-evidence request with three sample files attached and checked. 20
Figure 9. Customer support desk with two new requests. 20
Figure 10. Answering a request. The draft survives the automatic refresh. 21
Figure 11. After replying and raising the priority. 21
Figure 12. Safety ledger and shift handover. 22
Figure 13. Demo controls drawer. 22
Figure 14. Customer sign-in. 23
Figure 15. Home with a problem at the customer's Home. 23
Figure 16. Network page for Home. 23
Figure 17. Report a problem. 23
Figure 18. Assistant answering a question. 23
Figure 19. Voice assistant. 23
Figure 20. Help center. 24
Figure 21. A question expanded in place. 24
Figure 22. Searching for “slow data”. 24
Figure 23. No results. 24
Figure 24. Help article. 24
Figure 25. Article screenshot and actions. 24
Figure 26. Contact form with invalid input. 24
Figure 27. Contact form filled in. 24
Figure 28. Request sent. 25
Figure 29. After the care team replied. 25
Figure 30. My support requests. 25
Figure 31. The conversation. 25
Figure 32. Replying and closing. 25
Figure 33. Opening a reference that is not yours or does not exist. 25
Figure 34. Help center on a tablet, 768 pixels wide. The phone-width column stays centred; no horizontal scrolling. 26
Figure 35. Support desk at 1024 by 768 pixels. Columns wrap and the table scrolls inside its panel when needed; the page itself does not scroll sideways. 26

SECTION A

A. Introduction

A1 What RAN Sentinel is

RAN Sentinel is an AI co-engineer for a mobile network. It watches the radio access network, notices when a cell starts to degrade, works out why, proposes a safe fix and applies it only after the responsible team approves.

A mobile operator runs thousands of cells. When one of them slows down because a crowd arrives, a tilt was set wrongly or a radio part fails, an engineer in the network operations centre (NOC) usually has to read alarms, open charts, guess at a cause and find the right team. RAN Sentinel automates the routine part of that work. AI agents built on Meta's Muse Spark models investigate with read-only tools, a verifier checks that every claim is backed by data, fixed safety rules and a digital twin test the proposed change, and only then does the recommendation reach a person.

The same system serves the operator's customers. In the Northstar Connect app a customer sees whether there is a problem where they live or work, when it should be fixed, and whether they are owed a credit. With this release the app also has a full Help & Support area: a searchable help center, troubleshooting articles with screenshots, a contact form, and support requests that the care team answers from the operations console.

A2 Who uses it

Role	Application and sign-in	What they do
NOC engineer	Operations console, `OPS_EMAIL`	Watches the network, opens incidents, reads the agents' evidence, answers field-evidence requests, signs off root causes and runs the demo controls.
Department approver	Operations console, same account in the demo	Approves, rejects or questions the change proposed for their team (RAN Optimisation, RF Engineering, Field Operations and others).
Care agent	Operations console, Customer support module	Reads customer support requests, replies, sets status and priority. In the demo this is the engineer account Sarah Chen.
Customer	Northstar Connect, `CUSTOMER_EMAIL`	Checks service, reports problems, manages settings and bill, talks to the assistant, reads help articles and sends support requests.
Judge or evaluator	Both, with the demo logins	Follows Section G to see the important features in five to ten minutes.

The two applications use separate sessions. A customer session cannot call any operations or support-desk endpoint, and an engineer session cannot read a customer's requests through the customer endpoints; Section B5 explains how this is enforced.

A3 What is real and what is simulated

The network, the subscriber, the bill and the operator are simulated so that anyone can run the project on a laptop. The software around them is real: the detector, the agents and verifier, the safety rules, the M-Plane change package with rollback, the customer care logic and the support ticket system all run exactly as described here, and their behaviour is covered by automated tests. When a Meta API key is present, the agents call Meta's models; without one, an offline rules engine takes over and every screen says so. The support module itself uses no AI at all: requests are answered by a person.

A4 Feature overview

Area	Feature	Where
Monitoring	Network overview with KPI tiles and cluster map, X.733-style alarm monitor, cell explorer with KPI charts and radio configuration	Ops: Monitor
Investigation	Incident workspace with timeline, verified diagnosis, plans and safety checks, evidence, field photos and files, agent console, report	Ops: Incidents
Decisions	One card per department with queue and pending decisions; root-cause sign-off and dispute	Ops: Approvals
Audit	Evidence-to-Action Safety Ledger, shift handover, model card	Ops: Assure
Support desk (new)	List, filter and search customer requests; read the conversation; reply; change status and priority; see the customer's other requests	Ops: Customer support
Customer status	Proactive alerts for Home, Work and Gym, repair tracker, recovery estimate, map and signal check	Connect: Home, Network
Self-service	Report a problem, settings, places, bill and outage credits, notifications	Connect: Account, Bill
AI care	Text assistant (Care Concierge) and hands-free voice agent; every change needs the customer's confirmation	Connect: Assistant, Ask
Help & Support (new)	Help center with search and topics, 17 articles with steps and real screenshots, contact form with validation, My support requests, ticket conversation with reply and close	Connect: Help

SECTION B

B. Architecture

B1 System overview

RAN Sentinel is one Python process. A FastAPI server serves both web applications as static files and exposes a JSON API under `/api`. A background thread advances the simulated network every few seconds; each tick produces new KPI readings, the detector scores them, and the orchestrator moves incidents through their workflow. The browser applications are plain JavaScript with no build step; they poll the API and patch the page in place, so the screen updates without flicker or lost input.

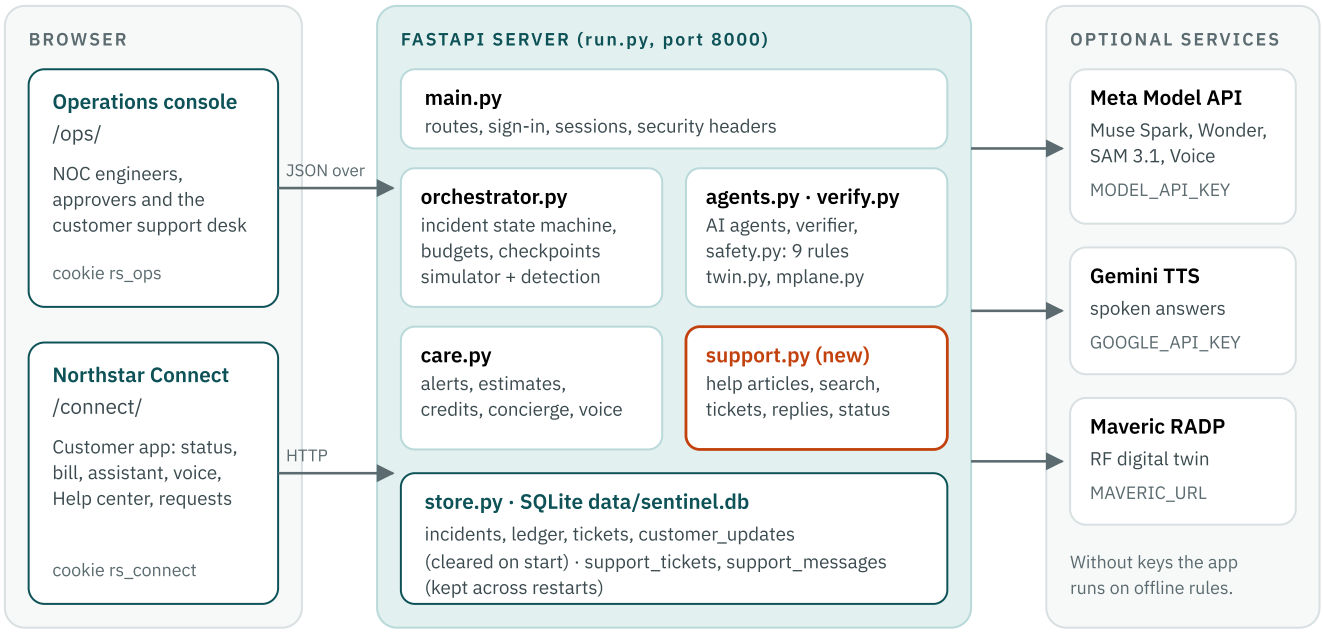


Diagram 1. Components and data flow. The support module (orange outline) is the part added in this release. External services are optional.

B2 Components

File	Responsibility
`run.py`	Starts Uvicorn on 127.0.0.1:8000.
`backend/app/main.py`	All HTTP routes, sign-in, sessions, lockout, security headers, the background tick loop.
`backend/app/orchestrator.py`	The incident state machine, budgets, checkpoints and resume, field-evidence pause, department tickets.
`simulator.py`, `detection.py`, `radio.py`	Simulated cells and KPIs, the threshold and Isolation Forest detector, the radio model for maps and signal.
`agents.py`, `verify.py`, `safety.py`, `twin.py`, `mplane.py`	AI agents and their prompts, the diagnosis verifier, nine deterministic plan rules, the digital twin, the NETCONF change package.
`care.py`	Customer alerts, recovery estimates, credits, reports, the Care Concierge and the voice agent.
`support.py` (new)	Help articles and search, ticket validation, ticket lifecycle, ownership checks, staff actions.
`store.py`	SQLite access with a lock; one method per read or write.
`frontend/ops/`, `frontend/connect/`, `frontend/shared/`	The two web applications and shared helpers (API calls, in-place page patching, live polling, busy buttons).

B3 Database

All data lives in one SQLite file, `data/sentinel.db`, which the server creates on first start; there is nothing to install or migrate by hand. The simulation tables (`incidents`, `ledger`, `tickets` for department decisions, and `customer_updates`) are cleared each time the server starts, because the simulated network also starts fresh. The two support tables are deliberately kept: a customer's requests and the care team's replies survive restarts and the demo reset. To start with an empty support desk, stop the server and delete `data/sentinel.db`.

Table	Columns	Notes
`support_tickets`	`seq` (integer key), `id` (CS-100001 and up, unique), `owner` (sign-in email), `status`, `category`, `priority`, `created`, `updated`, `data` (JSON with name, reply email, subject, assignee, unread flags, first response time)	Indexed by owner. IDs are sequential; access is checked on every read, so a guessed ID reveals nothing.
`support_messages`	`seq`, `ticket_id`, `role` (customer, agent or system), `author`, `body`, `created`	Status and priority changes are stored as system messages, so the conversation is also the history.
`incidents`, `ledger`, `tickets`, `customer_updates`	Incident snapshots, the append-only safety ledger, department decisions, customer-facing network updates	Cleared on start. See `docs/API_AND_DATABASE.md`.

B4 API flow for a support request

Diagram 2 follows one request from the contact form to the care team and back. The customer's browser posts the form; the server validates every field, sets the owner to the signed-in email (never to anything the browser sends), stores the ticket and its first message in one transaction and returns the reference. The support desk lists tickets with a new-message marker. When the agent replies, the reply and a status note are stored and the ticket is flagged unread for the customer, whose open conversation refreshes every five seconds.

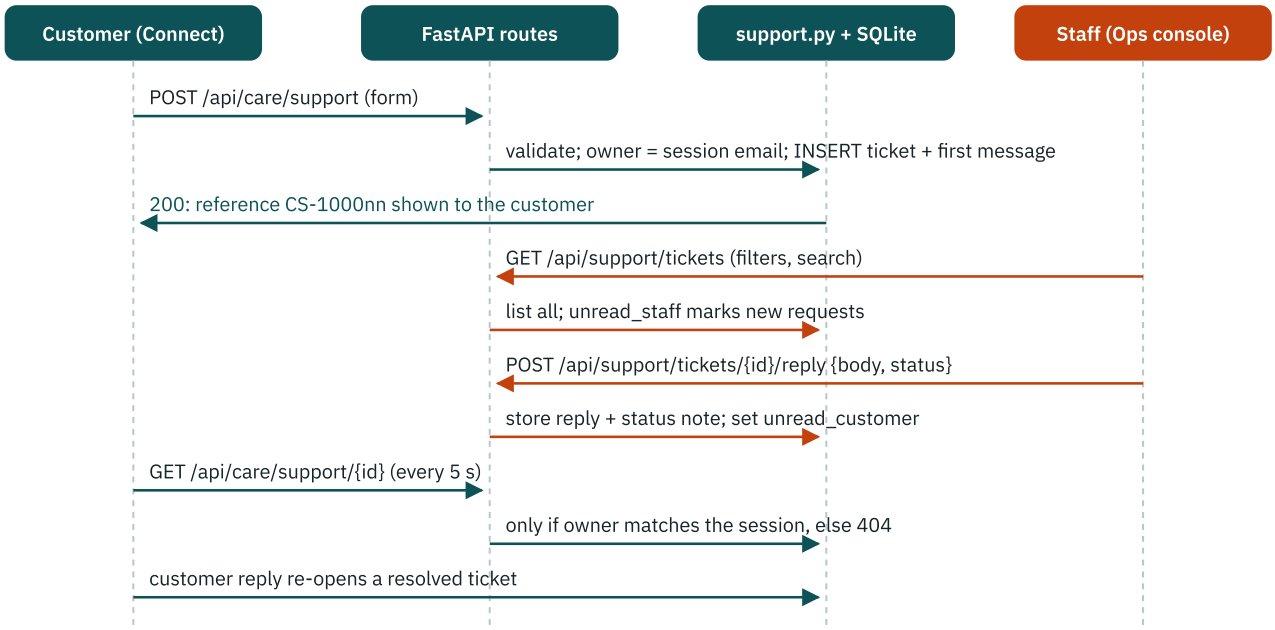


Diagram 2. Sequence of a support request. Teal arrows are customer calls, orange arrows are staff calls.

Endpoint	Session	Purpose
`GET /api/care/help?q=&category=`	Customer	Help topics and articles, ranked search
`GET /api/care/help/{id}`	Customer	One article with related articles
`GET /api/care/support` · `POST /api/care/support`	Customer	My requests · create a request
`GET /api/care/support/{id}`	Customer	One request with its conversation (owner only)
`POST /api/care/support/{id}/reply` · `/close`	Customer	Reply (re-opens a resolved request) · close
`GET /api/support/tickets?status=&category=&q=`	Staff	All requests with counts and unread total
`GET /api/support/tickets/{id}`	Staff	Ticket, conversation, the customer's other requests

Endpoint	Session	Purpose
`POST /api/support/tickets/{id}/reply` · `/update`	Staff	Reply and set the next status · change status or priority

B5 Authentication and access control

Each application has its own sign-in. `POST /api/login` checks the email and password against the values in `.env` with a constant-time comparison and, on success, sets an HttpOnly, SameSite=Lax cookie: `rs_ops` for the console or `rs_connect` for the customer app. The session records which portal it belongs to, and every protected route checks that portal, so a customer cookie presented to an operations route is refused with 401 even if it is copied into the other cookie name. Five failed sign-ins from one address lock sign-in for five minutes. Responses carry `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` and a same-origin referrer policy.

For support requests the server uses the session's email as the owner. A customer can list and open only their own requests; asking for anyone else's returns 404, the same answer as a request that does not exist, so the endpoint cannot be used to discover other customers' references. Status and priority can only be changed through staff endpoints. All text is cleaned of control characters, limited in length and stored as typed; both applications escape it when they display it, so a message containing HTML is shown as text and never runs. Sessions are held in memory and end when the server restarts; this is a demo design, and production use would need a real identity provider (see Section H).

SECTION C

C. Installation and setup

A clean install takes about five minutes. You need Python and Git; everything else is optional.

C1 Requirements

Item	Needed?	Notes
Python 3.10 or newer	Required	Windows: install from python.org and tick "Add python.exe to PATH". The `py` launcher is used below.
Git	Required	To clone the repository. You can also download the ZIP from GitHub.
A modern browser	Required	Chrome, Edge or Firefox. The voice agent needs microphone permission.
Meta Model API key	Optional	Without it the agents use the offline rules engine and say so on screen.
Google (Gemini) API key	Optional	Natural voice for spoken answers; without it the browser's own voice is used.
Node.js 18+ and Playwright	Optional	Only for the browser checks in Section F.
Docker	Optional	Only for the Maveric RF twin (`docs/MAVERIC_TWIN.md`).

C2 Get the code

Run in: any folder where you keep projects, for example D:\Projects

```
git clone https://github.com/AKMaasif/Meta-Ai-Global-Hackathon-Ran-Sentinel-.git
cd Meta-Ai-Global-Hackathon-Ran-Sentinel-
```

Git creates a folder named `Meta-Ai-Global-Hackathon-Ran-Sentinel-`. Every later command runs inside that folder, which this manual calls the **project folder**.

C3 Windows: one-click start

Double-click `start.bat` in the project folder. On the first run it creates a virtual environment in `.venv`, installs the Python packages from `requirements.txt`, copies `.env.example` to `.env`, starts the server and opens the operations console in your browser. Later runs skip the installation. Leave the black window open while you use the app; closing it stops the server.

C4 Step by step

If you prefer to see each step, or you are on macOS or Linux, run the commands yourself.

Run in: the project folder (Windows PowerShell)

```
py -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
py run.py
```

Run in: the project folder (macOS or Linux)

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python3 run.py
```

When the server is ready it prints `RAN Sentinel ready. AI engine: …`. If PowerShell refuses to run `activate`, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once and try again.

C5 Configuration

Settings live in `.env` in the project folder. The file is listed in `.gitignore` and must never be committed. Every setting has a working default, so the app starts with an untouched copy of `.env.example`.

Setting	Default	Purpose
`OPS_EMAIL` / `OPS_PASSWORD`	ops@northstar.demo / sentinel-demo	Demo sign-in for the operations console and support desk. Change both before showing the app on a shared network.
`CUSTOMER_EMAIL` / `CUSTOMER_PASSWORD`	customer@northstar.demo / connect-demo	Demo sign-in for Northstar Connect.
`MODEL_API_KEY`	empty	Meta Model API key. Paste it after the equals sign, with no quotes.
`GOOGLE_API_KEY`	empty	Gemini text-to-speech for the voice agent.
`MAVERIC_URL`	empty	Address of a running Maveric RADP service, for example http://localhost:8080.
`SIM_TICK_SECONDS`	3	Seconds between simulated 15-minute telemetry windows.
`PRIMARY_MODEL`, `FALLBACK_MODEL` and others	Muse Spark 1.3, 1.2, 1.1 …	Which Meta model each agent uses. Leave as they are unless told otherwise.

Restart the server after editing `.env` : press Ctrl+C in its window and start it again.

C6 Database setup

There is no separate database step. On first start the server creates `data/sentinel.db` and all tables, including the two support tables. Support requests are kept across restarts; the simulation is not. To reset everything, stop the server and delete `data/sentinel.db` ; it is recreated empty on the next start.

C7 First run and sign-in

Open http://127.0.0.1:8000/ops/ for the operations console and http://127.0.0.1:8000/connect/ for the customer app, and sign in with the demo accounts above. The two can be open at the same time in different tabs. The console status bar shows "Offline rules engine" or "Meta Model API", so you always know which engine produced a result.

C8 Optional components

Meta models. Put your key after `MODEL_API_KEY=` , restart, and run `py scripts\check_meta.py` in the project folder to confirm each model answers. **Voice.** Add `GOOGLE_API_KEY` for Gemini voices; the voice agent also works without it. **Maveric.** Follow `docs/MAVERIC_TWIN.md` , then set `MAVERIC_URL` . **Browser checks.** In the project folder run `npm i --no-save playwright` and `npx playwright install chromium` once.

C9 Check the installation

Run in: the project folder, with the virtual environment active

```
python -m pytest -q
# with the server running, in a second terminal:
node scripts/check_support_ui.js
```

A healthy install reports `177 passed` and `All 53 support UI checks passed.` Section F explains what these cover and how long they take.

SECTION D

D. User guide

This section explains each feature in the order a user meets it: first the operations console used by engineers and the care team, then the Northstar Connect app used by customers. Figure numbers point to the annotated screenshots in Section E.

D1 Signing in to the operations console

Open `http://127.0.0.1:8000/ops/` . The sign-in page asks for the engineer email and password from `.env` (Figure 1). After five wrong attempts from the same computer, sign-in is locked for five minutes and the page says how long to wait. If you enter the customer account here, the page tells you that this is a customer account and links to the customer app instead.

The console follows the layout of a network operations tool. A dark top bar holds the search box, live alarm counters by severity, the AI engine in use, the simulated clock and the signed-in user. The module rail on the left groups the pages into Monitor, Operate and Assure, with a badge where something waits for you. The status bar at the bottom shows when the screen last refreshed, the telemetry window, model calls and the twin and M-Plane links (Figure 2). Everything refreshes on its own every few seconds without losing what you are typing.

WHO	Engineers, approvers, care agents	**WHERE**	/ops/ → sign-in
YOU PROVIDE	OPS_EMAIL and OPS_PASSWORD	**YOU GET**	The network overview
IF IT GOES WRONG	Wrong password: red message under the form. Locked: wait the time shown. Customer account: link to the customer app.		

D2 Monitoring the network

Network overview is the home page. Five KPI tiles show availability, drop rate, PRB utilisation, throughput and SINR across the cluster with a twelve-hour trend. The cluster map shows the 12 sites and 36 sectors; zoom in, click a site for its cells and click a sector to select that cell. The work queue lists open incidents with their severity, workflow progress and owner. When no internet connection is available the map keeps the US state outlines and says the street map is unavailable, as in the screenshots in this manual.

Alarm monitor lists every alarm with X.733-style fields: severity, alarm ID, time raised, managed object, area, probable cause, event type and the linked incident. Use the severity and state buttons and the text filter to narrow the list, and Ack to acknowledge an alarm (Figure 3). Acknowledging records who saw the alarm; it does not clear it. **Cell explorer** shows the network tree by site on the left and, for the chosen cell, KPI charts against their limits, alarms, the radio configuration read over M-Plane and the detector scores (Figure 4).

D3 Following an incident

When the detector confirms a problem over two telemetry windows, it opens an incident and the agents start on their own. The **Incidents** page lists them; filter by open, closed or all and click a row to open it (Figure 5). The incident workspace starts with a fact bar (cell, area, band, opening time, owner, ticket) and the investigation workflow, eleven steps from Detector to Recovery, each with a short note on what happened (Figure 6).

The tabs below hold the detail. Diagnosis shows the KPI evidence, alarms, ranked root-cause hypotheses with the evidence for and against each, and the verifier's result. Plans and safety shows each proposed change, the nine safety rules and the twin's what-if. Evidence lists every read-only tool call. Agent console shows every model call with its prompt and answer, Report is the incident report, and Ledger is this incident's part of the audit trail. The right-hand column shows the agent's living investigation plan and, when a decision is needed, the decision card for the owning team.

WHO	NOC engineer	**WHERE**	Operate → Incidents → a row
YOU PROVIDE	Nothing to start; decisions and evidence when asked	**YOU GET**	Verified diagnosis, safe plan, decision card
IF IT GOES WRONG	If the Meta API fails, the incident shows a Resume button and continues from its last saved step. If the safety rules block every plan, the incident is escalated to a person.		

D4 Approvals, field evidence and sign-off

Approvals shows one card per team (RAN Optimisation, RF Engineering, Field Operations, NOC Data Platform, NOC Shift Lead and Regulatory Compliance) with what waits for each, and a table of decisions (Figure 7). Open a decision to approve, reject or ask for more information; a second-opinion review from a different model is shown with any concerns.

Approving a configuration change sends it to the simulated radio as an M-Plane change package; if the cell does not recover, the change rolls back by itself.

Sometimes the data cannot settle the cause, for example when the diagnostics service is down. The agent then pauses and asks a field engineer a specific question. The **Field evidence** card shows the question, a box for what was found and an optional file upload for photos, CSV logs, PDF, Word or text reports (Figure 8). Each file is read as soon as it is attached and marked as fitting, contradicting or unclear for the diagnosis. Sample files for every fault type are in `samples/field_evidence/`. Send the note and the investigation resumes.

D5 Customer support desk (staff)

The **Customer support** module, under Operate, is where the care team answers requests customers send from the Help center (Figure 9). The badge on the rail counts requests with a new customer message. At the top, four counters show active, unread, waiting-for-customer and resolved requests. The status buttons filter the list (Active is the default and covers Open, In progress and Waiting for customer), the category menu narrows it to one topic, and the search box finds a reference number, subject, customer name, email or any word in the conversation; press Enter to search. Unread requests are shown in bold with a red dot.

Click a request to open it (Figure 10). The conversation shows the customer's messages on the left, the care team's on the right, and every status or priority change as a small centred line, so it doubles as the history. To answer, type in *Message to the customer*, choose what the status should become (Waiting for customer, In progress or Resolved) and click **Send reply**. The ticket panel on the right shows the reference, the customer, the reply email, the account they are signed in with, the assignee and the time of the first response; use it to change status or priority and click **Save changes**. Below it you see the customer's other requests. A closed request cannot be answered until you reopen it with the status control.

WHO	Care agent (Sarah Chen in the demo)	**WHERE**	Operate → Customer support
YOU PROVIDE	A reply of up to 4,000 characters; status; priority	**YOU GET**	Reply visible to the customer within five seconds; status history kept
IF IT GOES WRONG	Empty reply: 'Write a message first.' Closed ticket: 'Reopen it before replying.' Unknown reference: 'Ticket not found' page with a link back.		

D6 Safety ledger, model card and demo controls

The **Safety ledger** is the append-only record of every detection, tool call, model output, safety check, decision and change, newest first, with a filter box. **Write shift handover** asks Muse Wonder to summarise all incidents and the ledger for the next shift (Figure 12). The **Model card** lists every agent, the Meta model behind it, its fallback and the measured accuracy of the detectors.

Demo controls, at the bottom of the rail, opens a drawer beside any page (Figure 13). Choose a cell, a problem and a severity and click **Create problem**; within about two telemetry windows an incident opens. Advance 15 min moves time forward, Clear all faults and Reset demo start over, and the failure switches show the safety paths: a red-team first plan, a twin outage, a diagnostics outage that triggers the field-evidence pause, and Meta API outages. Reset demo restarts the simulation but keeps support requests.

D7 Customer app: sign-in, home, network and reports

Open `http://127.0.0.1:8000/connect/` and sign in with the customer account (Figure 14). The app is built for a phone; on a tablet or computer it is shown as a centred phone-width column. Five tabs at the bottom lead to Home, Network, Account, Bill and Help, the bell at the top opens notifications, and the **Ask** button above the tabs opens the voice assistant.

Home leads with the most important thing: if there is a problem at one of the customer's places, a coloured card says what is happening, how far the repair has got and when service should be back (Figure 15). Below are Home, Work and Gym with their status, the balance and data use, and shortcuts. **Network** shows each place in detail with the repair tracker, the recovery estimate and its confidence, a map of nearby towers and a signal check (Figure 16). **Report a problem** asks where and what is wrong; if engineers are already fixing a problem there, the report is added to their repair, otherwise the app suggests checks for the phone (Figure 17).

D8 Account, bill and notifications

Account shows the lines and devices, the plan, settings switches (Wi-Fi Calling, Data Saver, network alerts, recovery updates) and the neighbourhood of each place; a change is saved immediately and confirmed with a short message. **Bill** shows the balance, the bill lines, payment history and outage credits: when a place has been down for 30 minutes or more, a $5 credit appears with an Apply button. A fixed rule decides this, not the AI. This is a demo carrier, so Pay now and AutoPay settings only say that no payment is taken. **Notifications** collects alerts, repair progress, restored messages and credit offers.

D9 The assistant and the voice agent

The **Assistant** page (Help → Ask the assistant, or Ask AI on Home) answers questions about the customer's places, bill and settings (Figure 18). Tap a suggestion or type a question. When an answer includes a change, such as turning on Wi-Fi Calling or applying a credit, it appears as a card with **Confirm** and **Not now**; nothing changes until the customer confirms. Under each answer, "How I answered" lists the steps, and a line says whether Muse Spark or the offline rules wrote it.

The **Ask** button opens the hands-free voice agent (Figure 19). Tap the microphone and speak, or type in the box. The agent shows what it heard, lists the steps it takes, answers aloud and can open pages for you. A change still needs a spoken "yes" in a later turn or a tap on Confirm.

D10 Help center and FAQs

The **Help** tab opens the Help center (Figure 20). It greets the customer by name and puts a search box first. Six quick options lead to the assistant, the contact form, My requests (with a count of active requests), Report a problem, Network status and Outage credits. Below them, six topics (Network and coverage, Billing and credits, Account and settings, Phone and device, Using the app and assistant, Something else) show how many articles each has. Tap a topic to list its questions; tap it again or *Show popular* to go back.

Each question expands in place to show the answer and a *Read the full article* button (Figure 21). Typing in the search box searches all 17 articles as you type, ranking title matches first (Figure 22); a search with no match says so and offers the assistant and the contact form (Figure 23). A full article has a topic label, a short summary, the explanation, numbered steps, a real screenshot of the relevant screen, buttons that open that screen, related articles and a *Didn't solve it?* box (Figures 24 and 25).

WHO	Customer	**WHERE**	Help tab
YOU PROVIDE	A search term, or a tap on a topic or question	**YOU GET**	Ranked articles with steps and screenshots
IF IT GOES WRONG	Search error: a message with Try again. Unknown article link: 'We couldn't find that article' with a link back to Help.		

D11 Contacting support and following a request

Contact support opens the form. Name and the email for replies are filled in from the account and can be changed; choose a category, write a subject of 4 to 120 characters and a description of 10 to 4,000 characters (a counter shows how many you have used). Tap **Send request**. If anything is missing or wrong, each field turns red with a message under it saying exactly what to fix, and the cursor moves to the first one (Figure 26). The server checks the same rules again, so the form cannot be bypassed. What you type is kept if you leave the page and come back.

When the request is accepted, a confirmation shows the reference (for example `CS-100001`), the subject, category and status, with buttons to view the request or go back to Help (Figure 28). **My support requests** lists active requests first and resolved or closed ones below, each with its reference, category, last update and status. A request with an unanswered reply from the care team shows *New reply*; the Help tab gets a red dot and the Help center shows a *The care team replied* card (Figures 29 and 30).

Opening a request shows its status, a banner when the care team is waiting for you or has marked it resolved, and the whole conversation including status changes (Figure 31). The page refreshes every five seconds, so a reply appears without reloading. Write in *Your reply* and tap **Send reply**; replying to a resolved request opens it again. **Close request** asks for confirmation first (Figure 32). A closed request cannot be answered; start a new one instead.

Status	Meaning for the customer	Who sets it
Open	The care team has the request but has not started.	New request; customer reply
In progress	Someone is working on it.	Care agent
Waiting for your reply	The care team asked a question. Staff see "Waiting for customer".	Care agent (default after a reply)
Resolved	The care team believes it is fixed; a reply re-opens it.	Care agent
Closed	The conversation has ended; no more replies.	Customer, or care agent

WHO	Customer	**WHERE**	Help → Contact support; Help → My requests
YOU PROVIDE	Name, email, category, subject, description; replies	**YOU GET**	Reference number, conversation with the care team
IF IT GOES WRONG	Invalid fields are highlighted. More than 20 open requests: asked to reuse or close one. Someone else's or unknown reference: 'We couldn't open that request'.		

SECTION E

E. Annotated screenshots

All screenshots below were taken from the running application with a headless Chromium browser on 1 November 2026, using the offline rules engine (no Meta key) and a fresh database. The console was captured at 1440 by 900 pixels and the customer app at phone size, 390 by 844. Orange outlines and numbers match the notes under each picture.

E1 Operations console

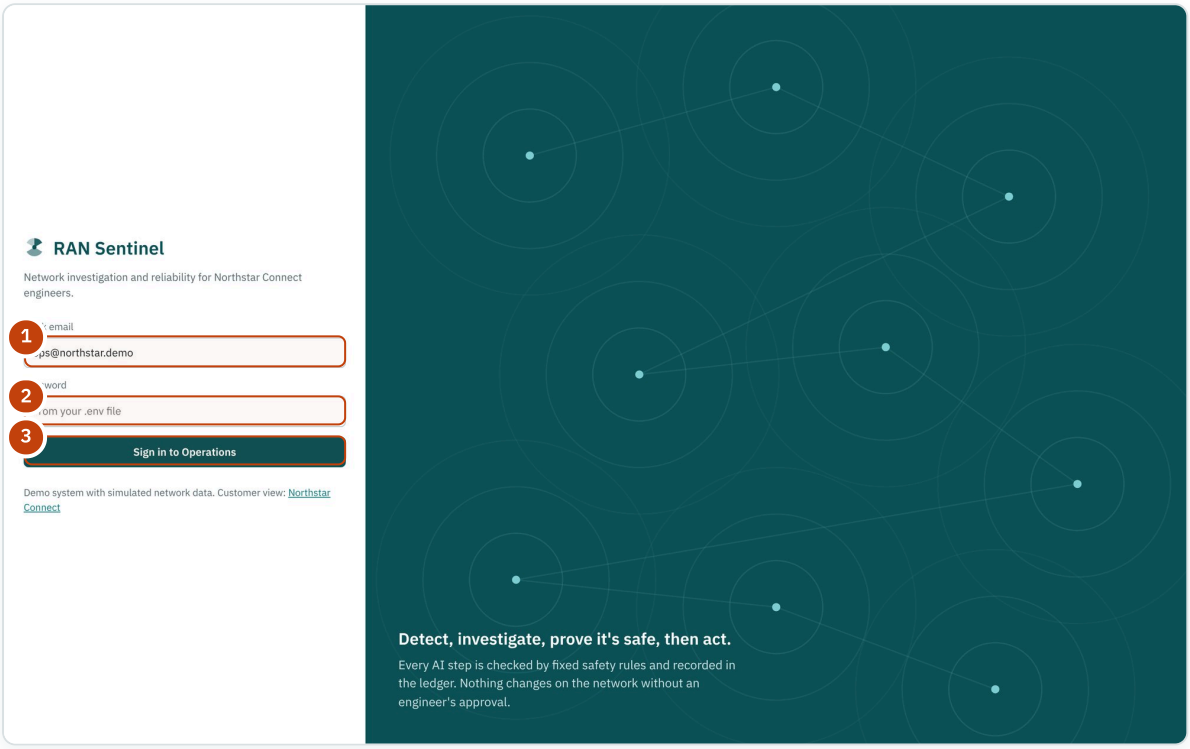


Figure 1. Operations console sign-in.

1. Engineer email (`OPS_EMAIL`)
2. Password (`OPS_PASSWORD`)
3. Sign in

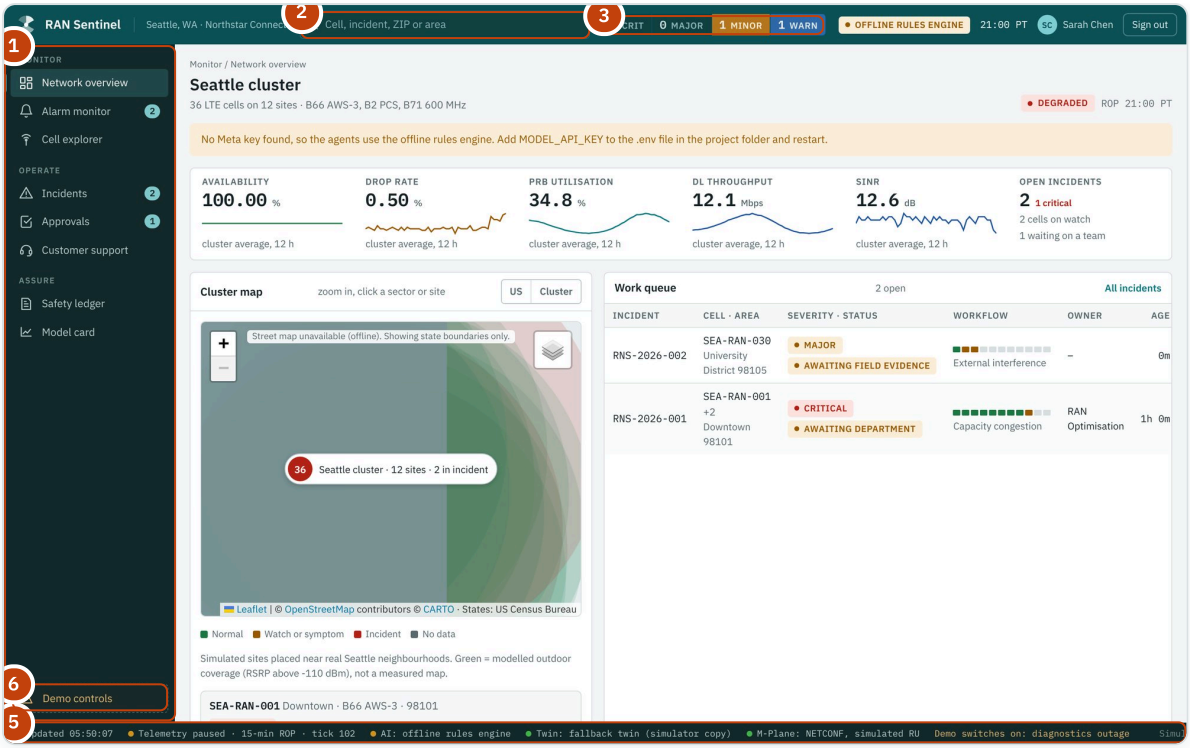


Figure 2. Network overview after two demo faults were created.

1. Module rail; badges count items waiting
2. Search by incident, cell, ZIP or area
3. Live alarm counters by severity
5. Status bar: refresh time, telemetry, AI engine, twin, M-Plane
6. Demo controls drawer

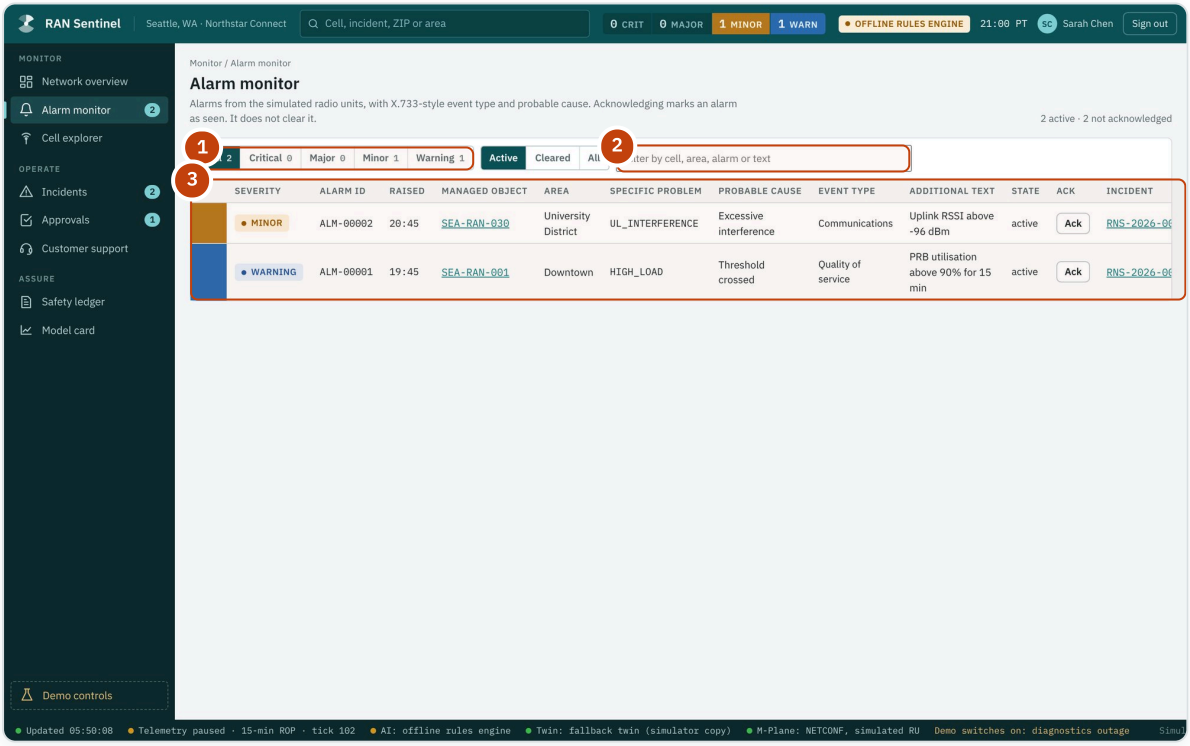


Figure 3. Alarm monitor.

1. Severity and state filters
2. Text filter
3. Alarms with X.733-style fields and Ack

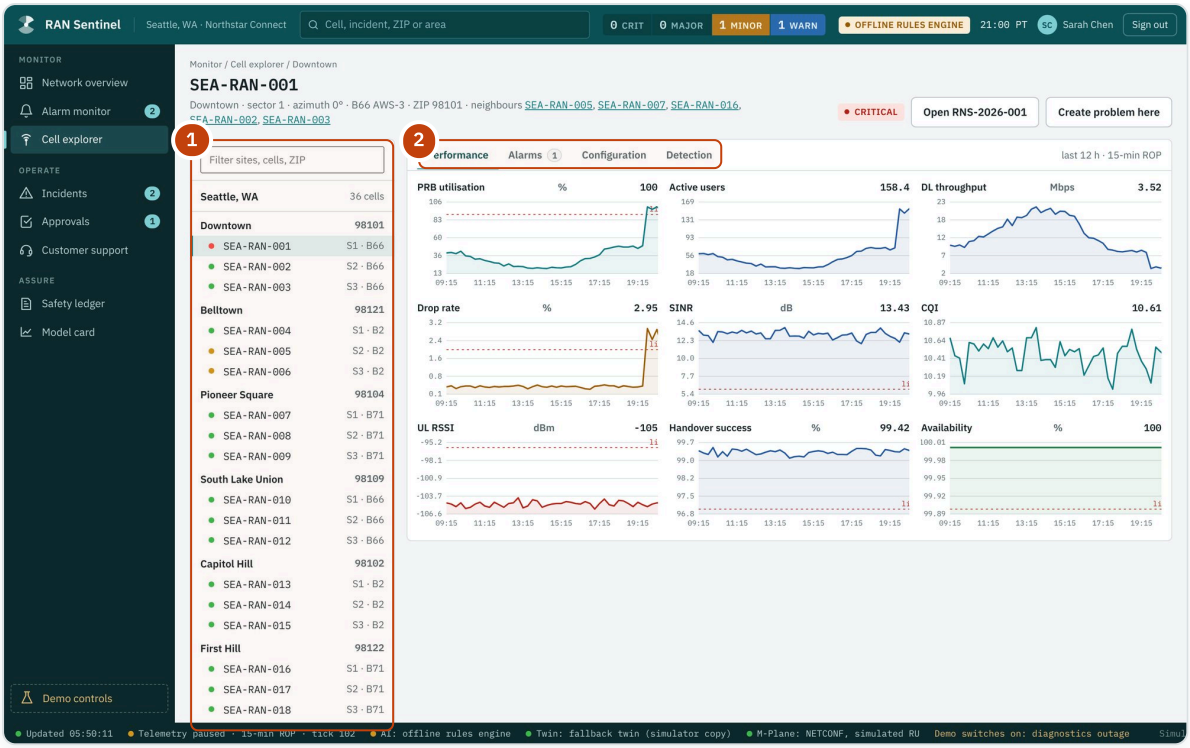


Figure 4. Cell explorer for SEA-RAN-001.

1. Network tree by site, with filter
2. Performance, alarms, configuration and detection tabs

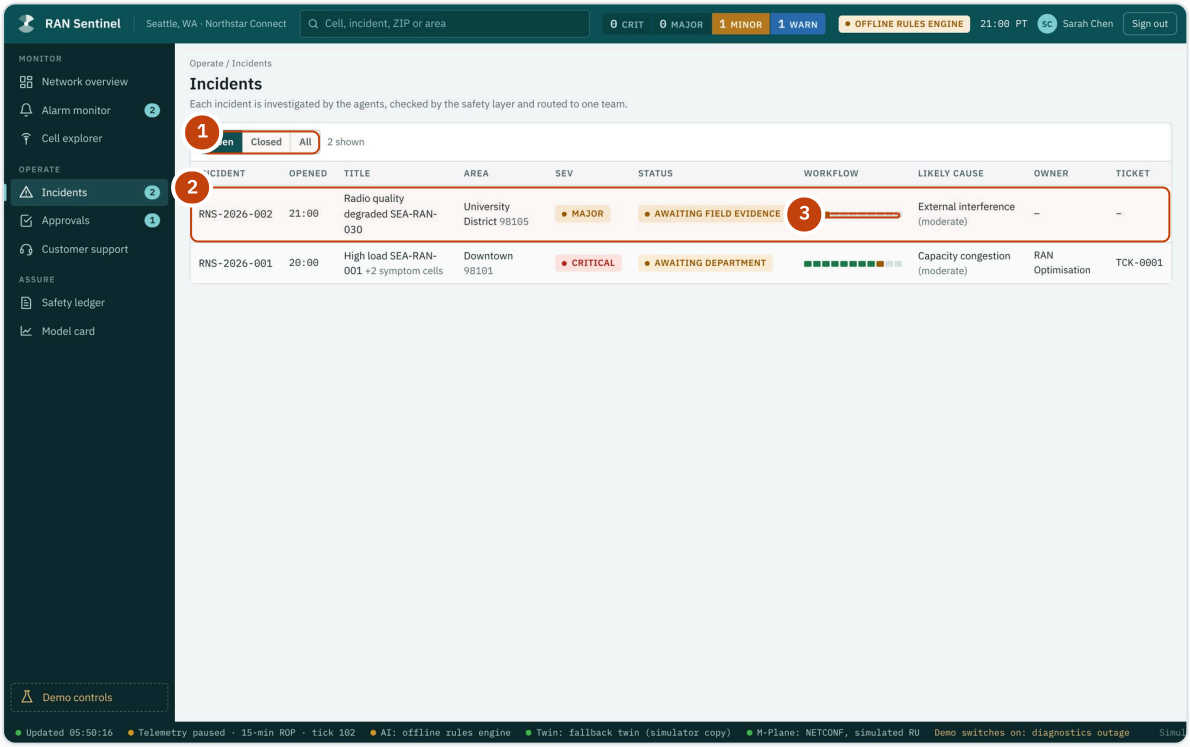


Figure 5. Incident list.

1. Open, closed or all
2. Click a row to open the workspace
3. Workflow progress at a glance

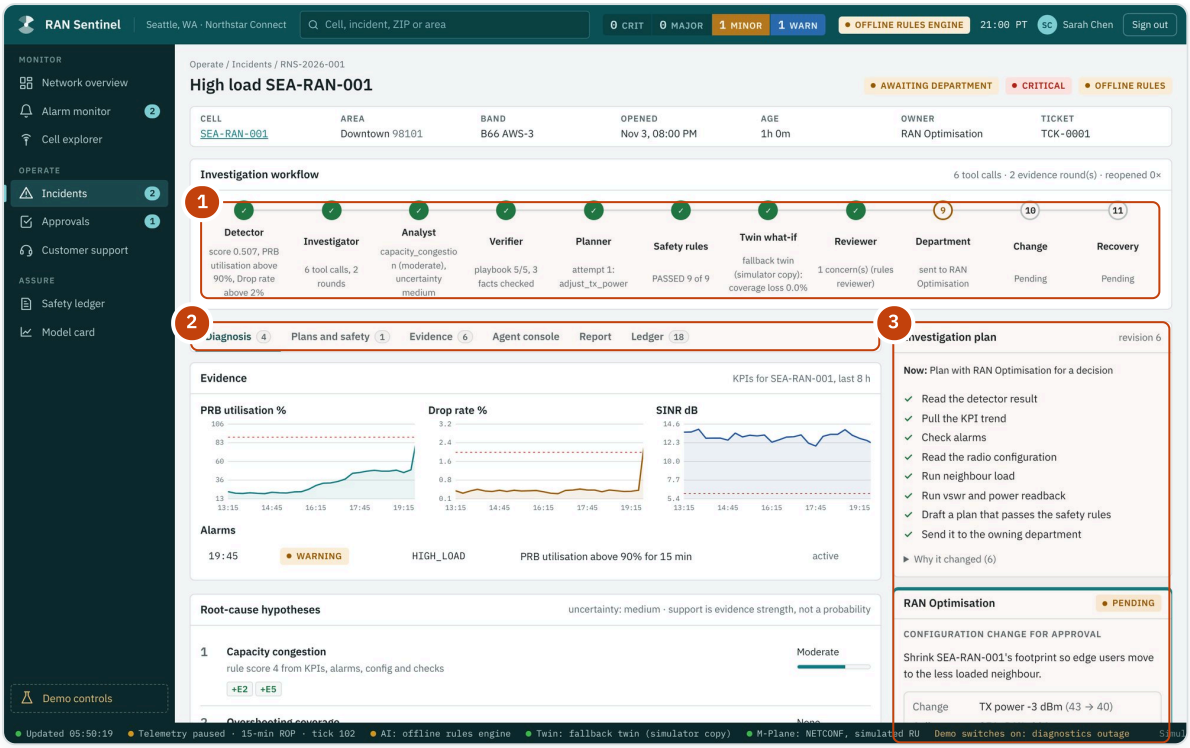


Figure 6. Incident workspace for a capacity problem awaiting the RAN Optimisation decision.

1. Investigation workflow, eleven steps
2. Detail tabs
3. Living investigation plan and decision card

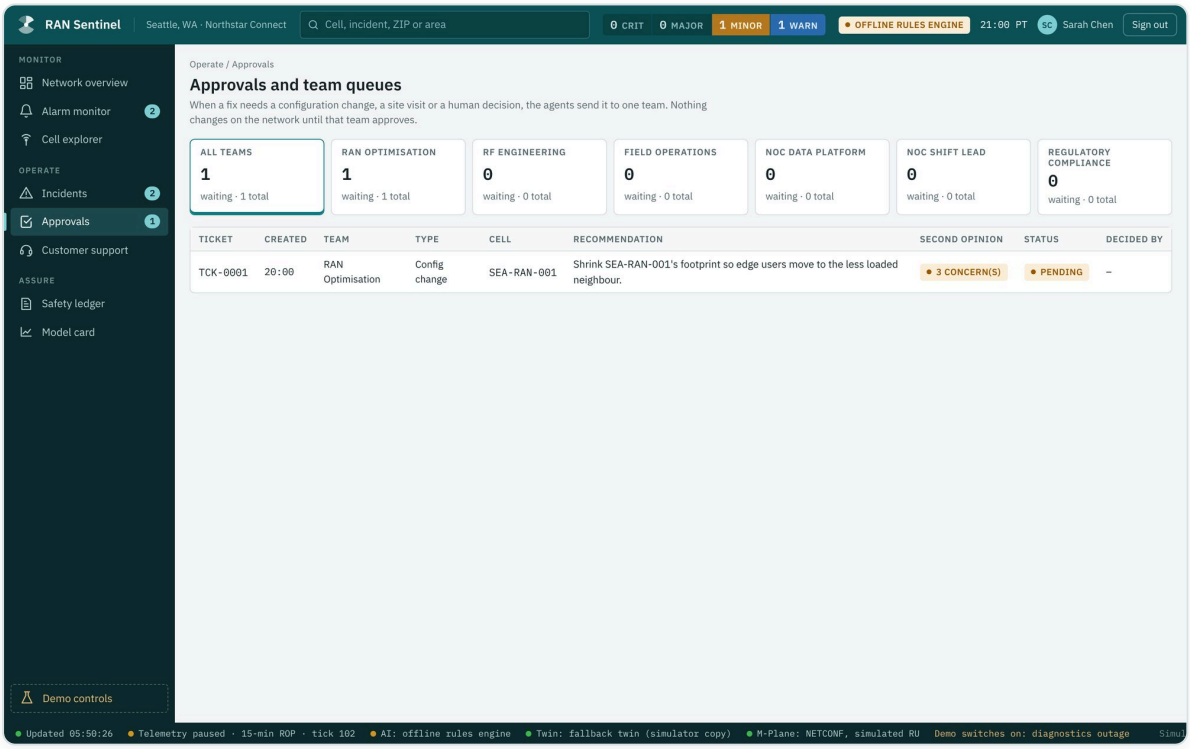


Figure 7. Approvals: one card per team and the decisions waiting for each.

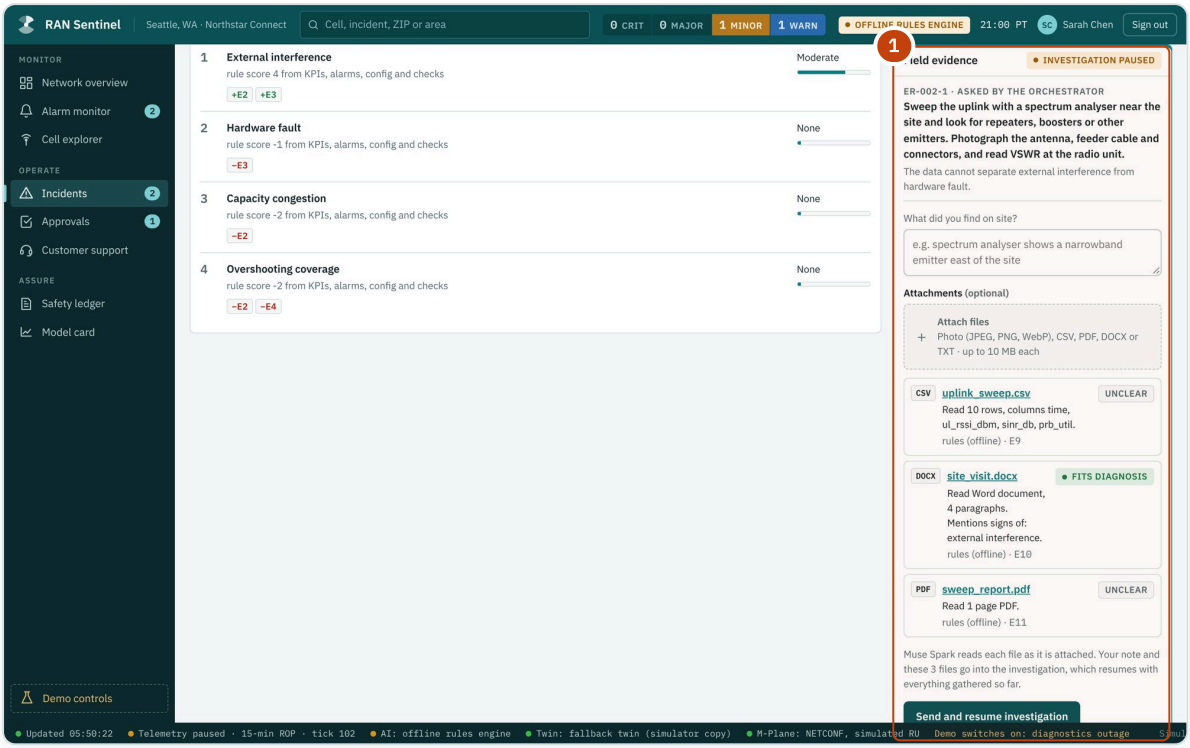


Figure 8. Field-evidence request with three sample files attached and checked.

1. The question for the field engineer, a note box and file upload; each file is marked as fitting, contradicting or unclear

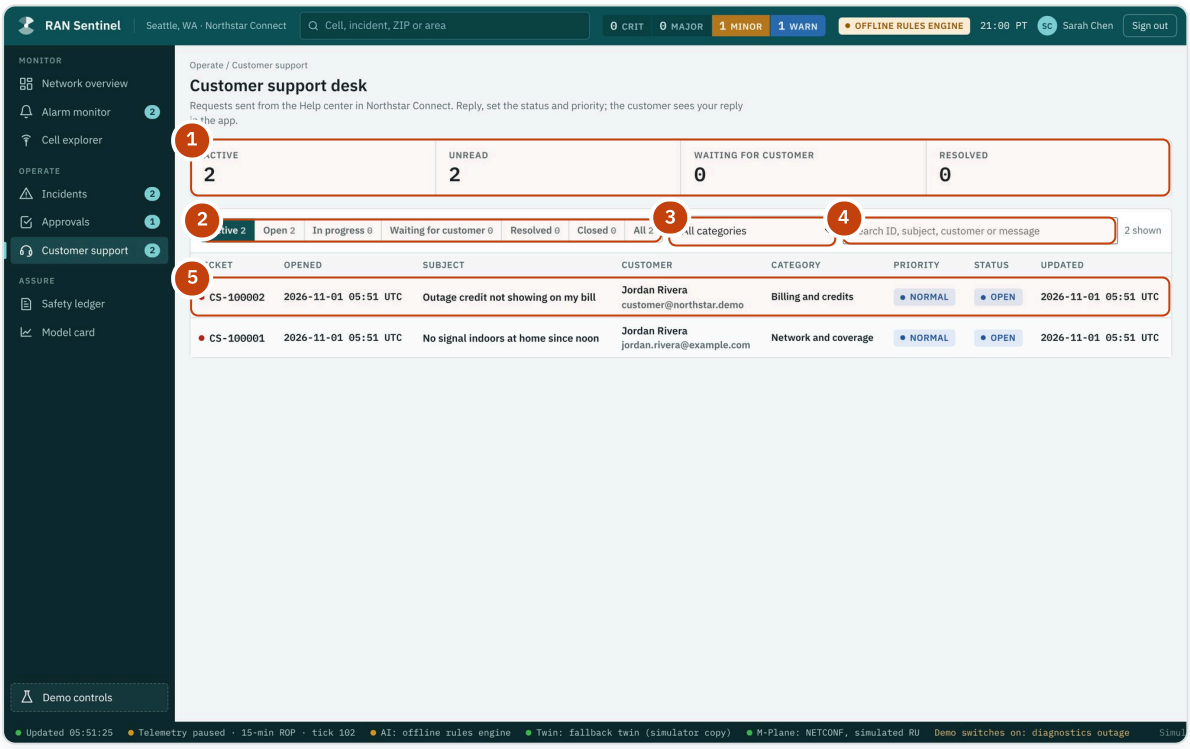


Figure 9. Customer support desk with two new requests.

1. Counters: active, unread, waiting, resolved
2. Status filter with counts
3. Category filter
4. Search reference, subject, customer or message
5. Unread request: bold with a red dot

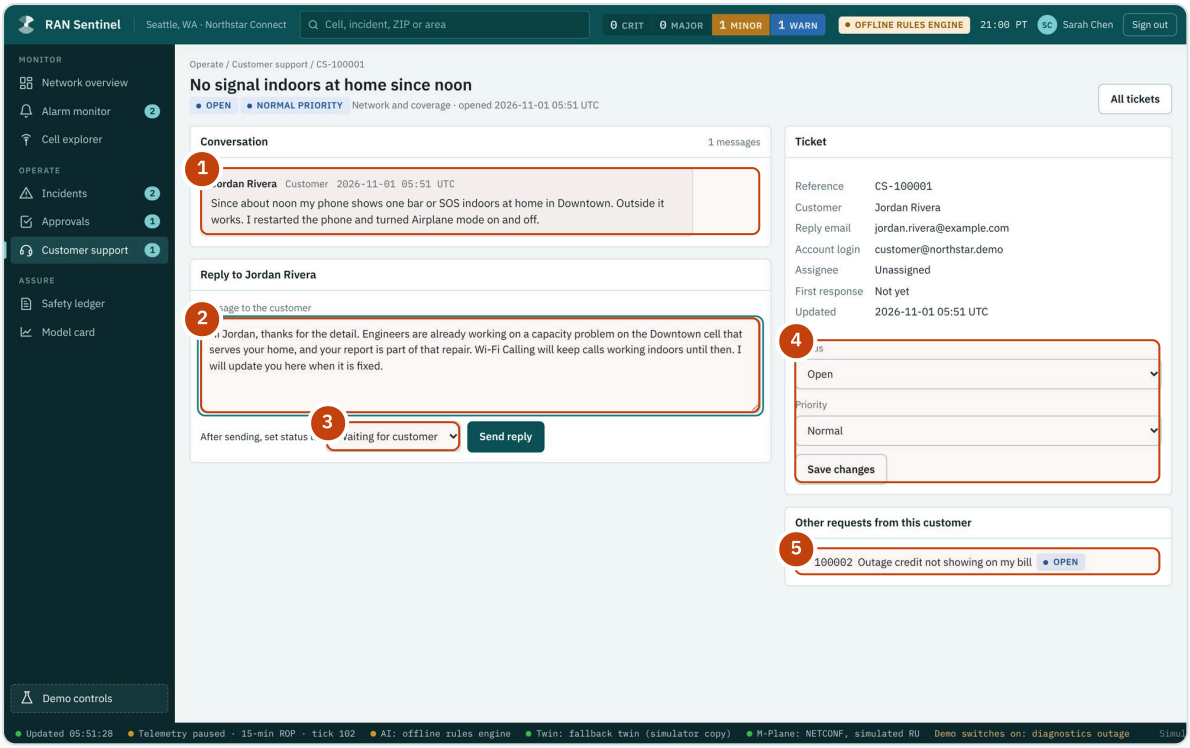


Figure 10. Answering a request. The draft survives the automatic refresh.

1. Conversation and status history
2. Message to the customer
3. Status after sending
4. Status and priority controls
5. The customer's other requests

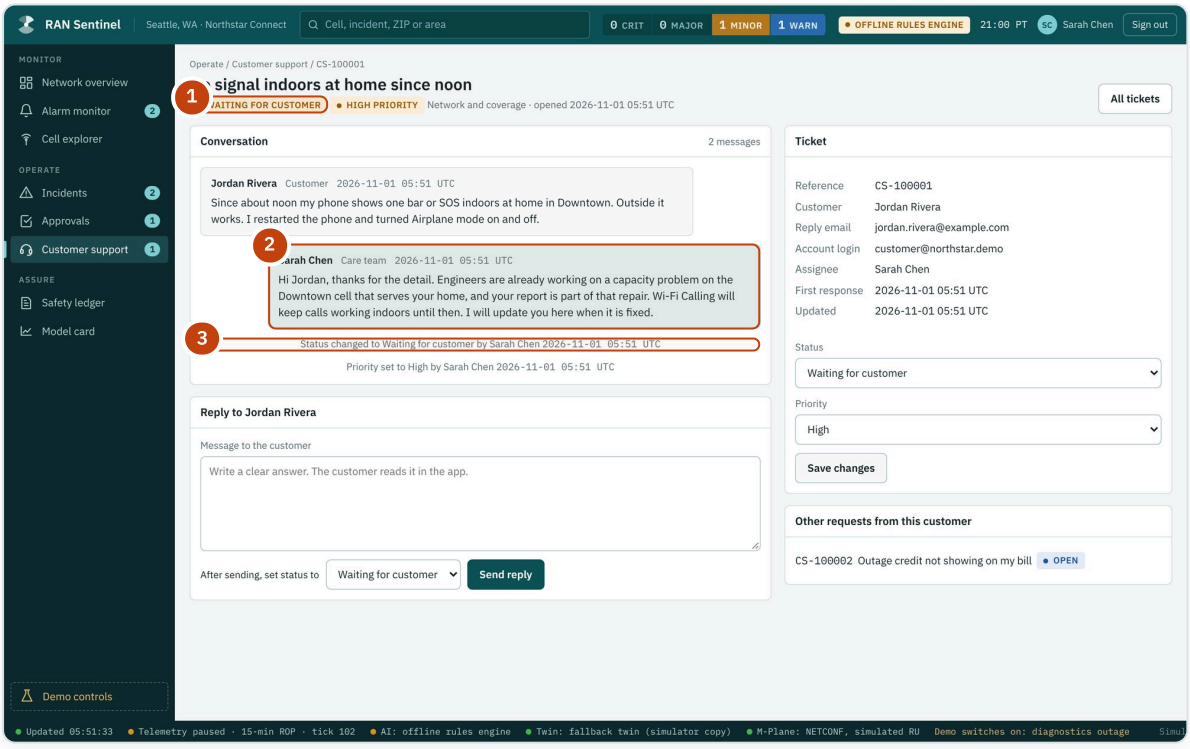


Figure 11. After replying and raising the priority.

1. New status and priority
2. The reply as the customer will see it
3. Status and priority changes recorded in the history

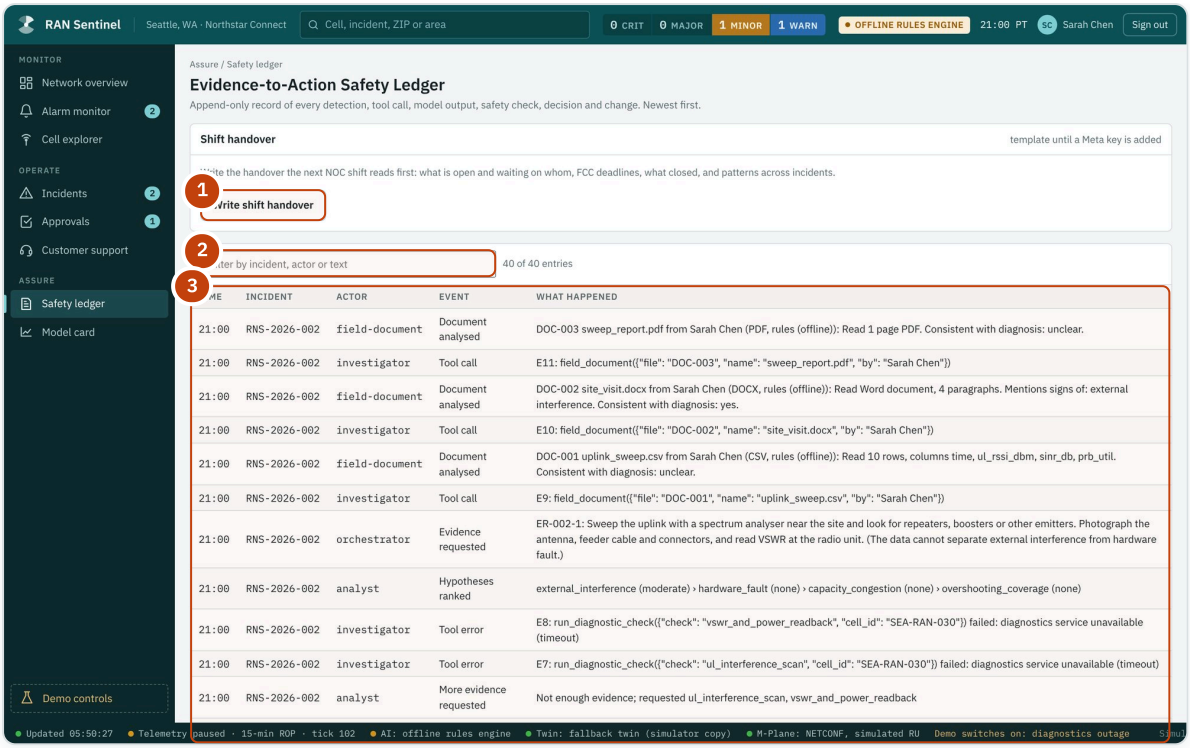


Figure 12. Safety ledger and shift handover.

1. Write the shift handover with Muse Wonder
2. Filter the ledger
3. Every action, newest first

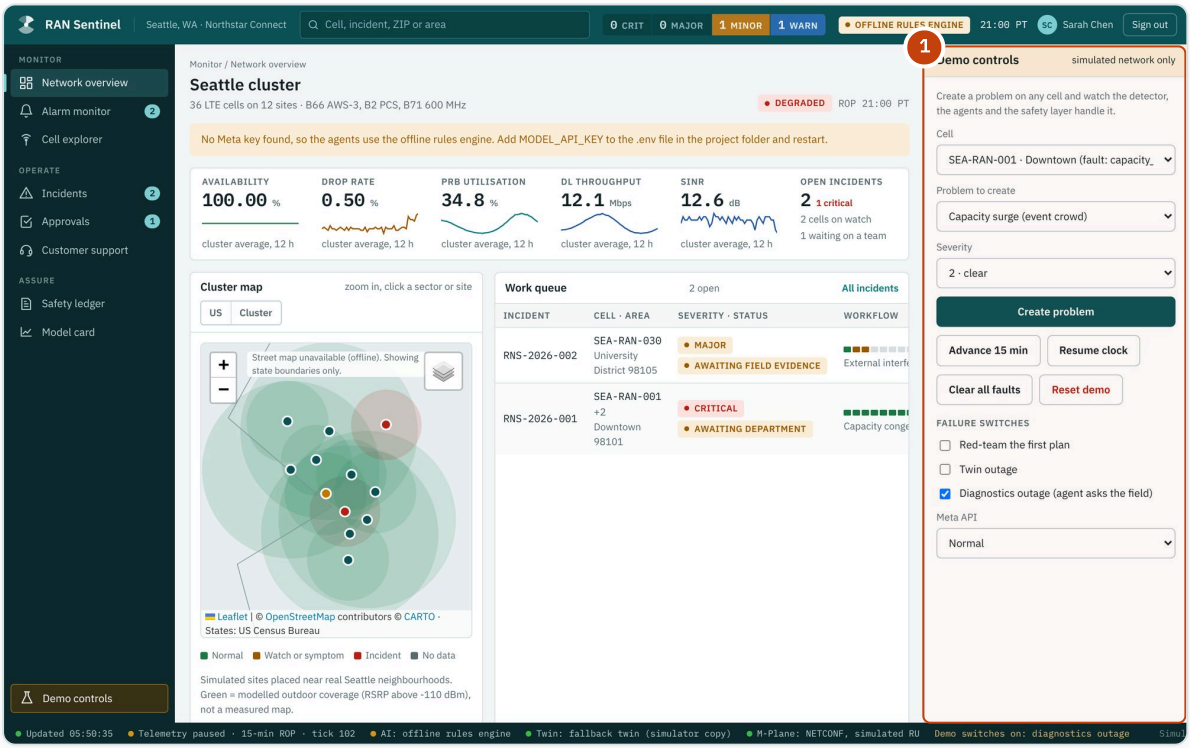


Figure 13. Demo controls drawer.

1. Create a problem, move time, reset, and switch failure paths

E2 Northstar Connect

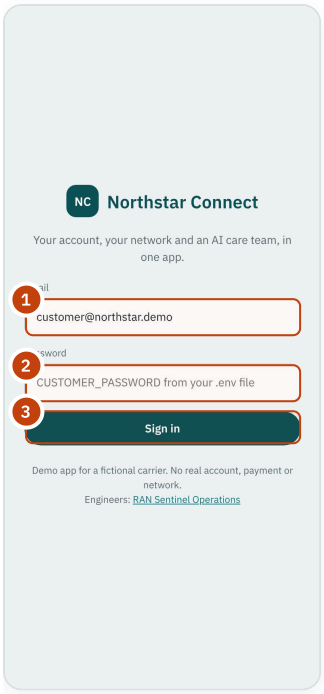


Figure 14. Customer sign-in.

1. Customer email
2. Password
3. Sign in

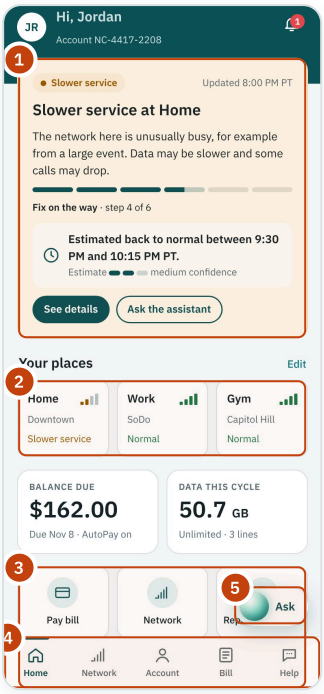


Figure 15. Home with a problem at the customer's Home.

1. Problem card with stage and estimate
2. Status of each place
3. Shortcuts
4. Tabs
5. Voice assistant

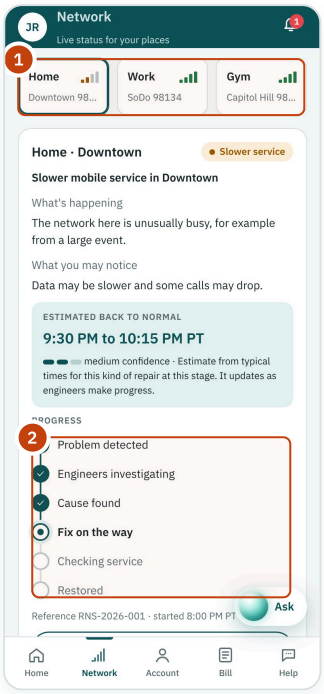


Figure 16. Network page for Home.

1. Choose a place
2. Repair tracker

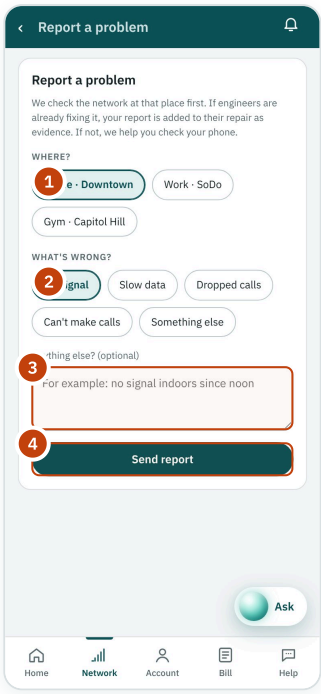


Figure 17. Report a problem.

1. Where
2. What is wrong
3. Optional detail
4. Send report

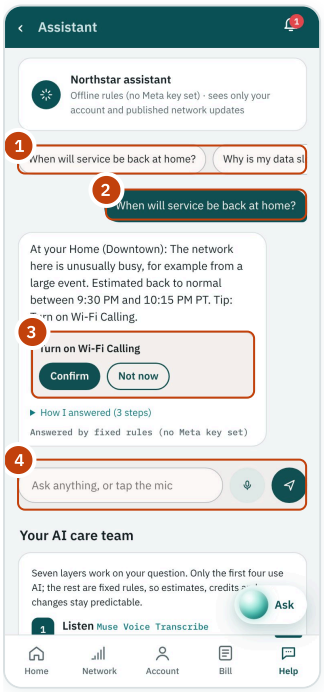


Figure 18. Assistant answering a question.

1. Suggested questions
2. The customer's question
3. A change waiting for Confirm
4. Type or speak

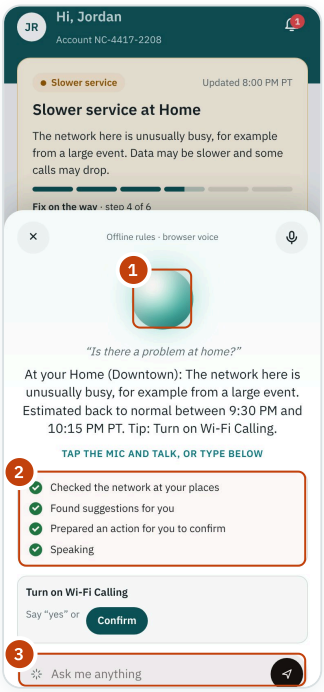


Figure 19. Voice assistant.

1. Listening orb
2. Steps taken
3. Type instead of speaking

E3 Help & Support

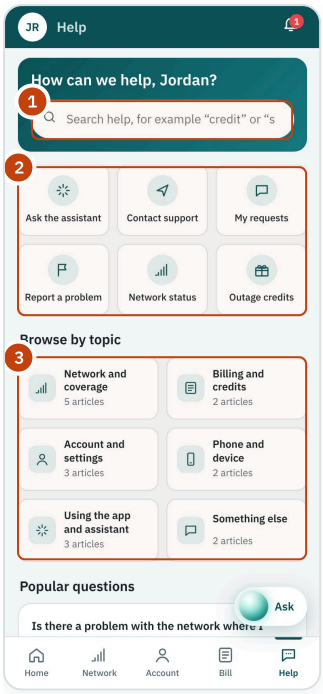


Figure 20. Help center.

1. Search help
2. Quick options
3. Browse by topic

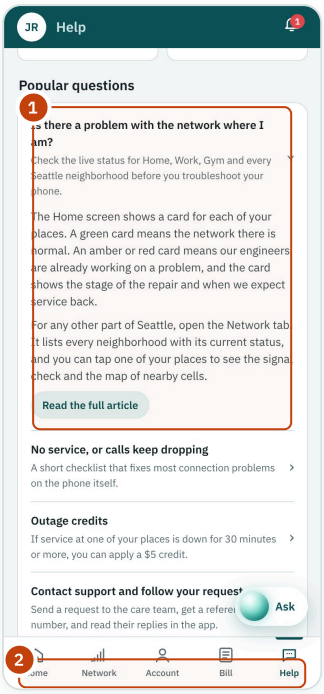


Figure 21. A question expanded in place.

1. Answer with a link to the full article
2. Still need help?

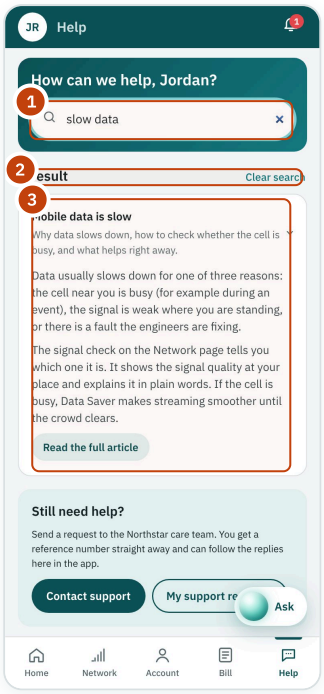


Figure 22. Searching for “slow data”.

1. Search box with clear
2. Result count and Clear search
3. Best match

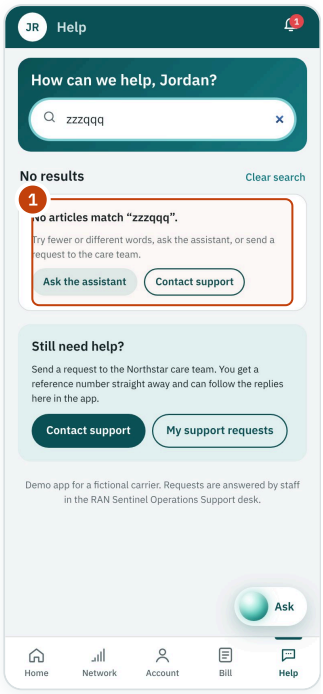


Figure 23. No results.

1. Empty state with the next steps

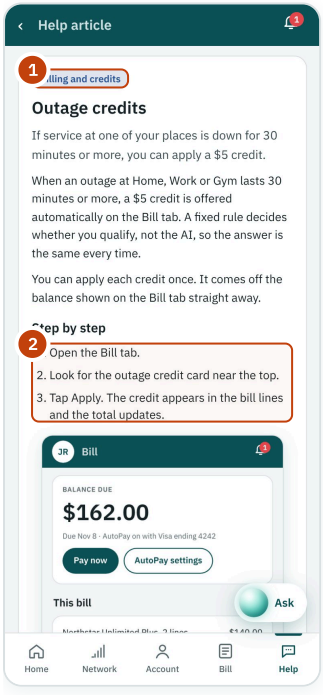


Figure 24. Help article.

1. Topic
2. Numbered steps

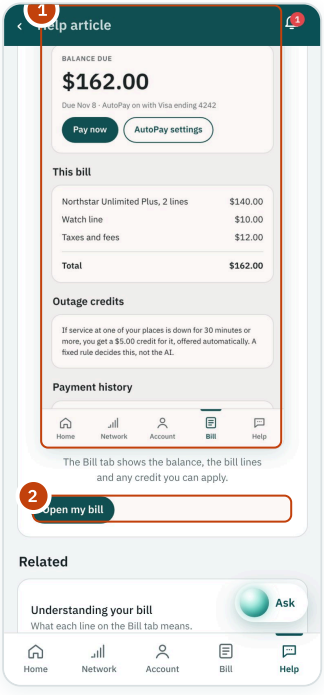


Figure 25. Article screenshot and actions.

1. Real screenshot of the screen described
2. Buttons that open that screen

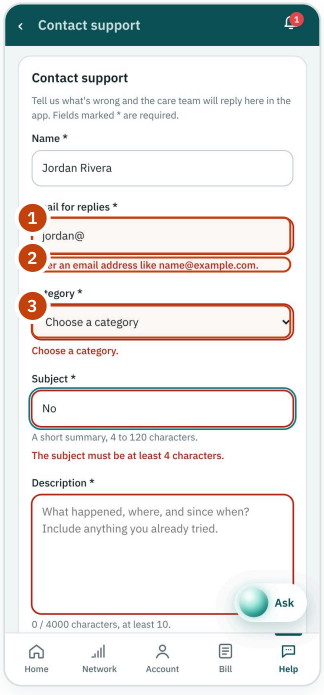


Figure 26. Contact form with invalid input.

1. Field marked invalid
2. What to fix
3. Required choice

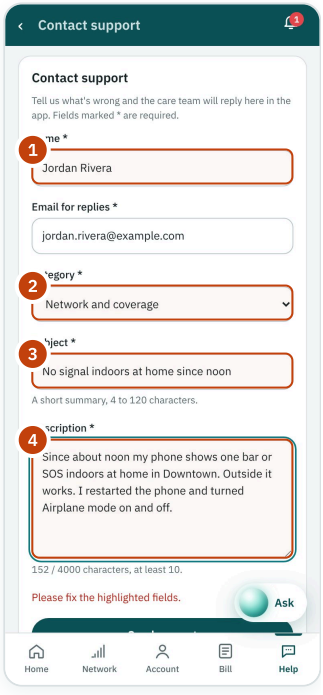


Figure 27. Contact form filled in.

1. Name and reply email from the account
2. Category
3. Subject
4. Description with counter

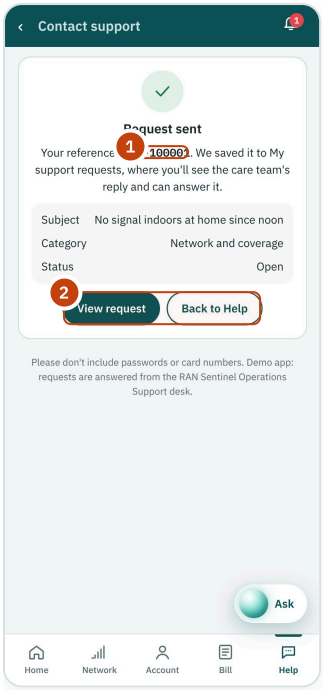


Figure 28. Request sent.

1. Reference number
2. View request or back to Help

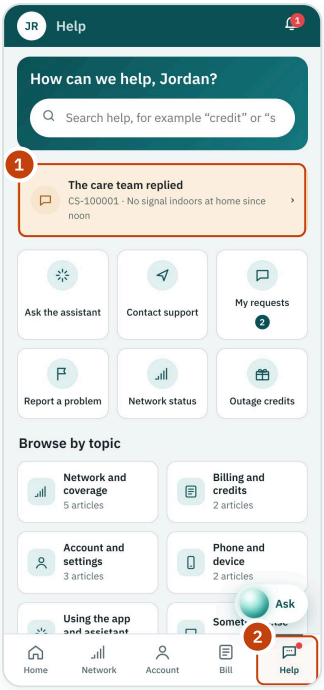


Figure 29. After the care team replied.

1. Reply card in the Help center
2. Red dot on the Help tab

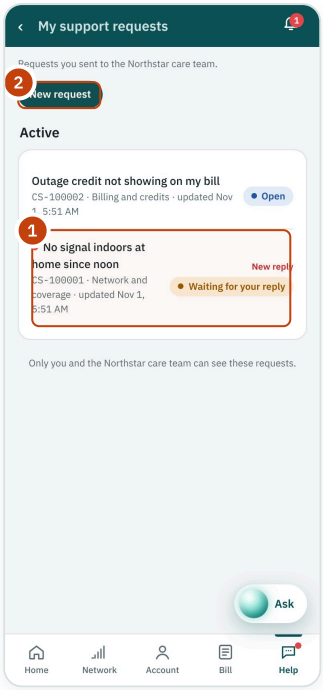


Figure 30. My support requests.

1. Request with a new reply
2. New request

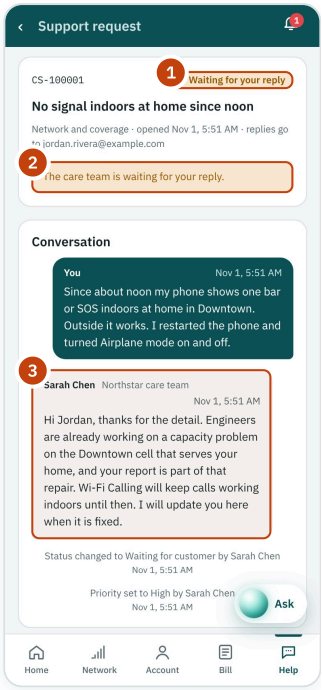


Figure 31. The conversation.

1. Status
2. Banner when the care team waits for you
3. The care team's reply

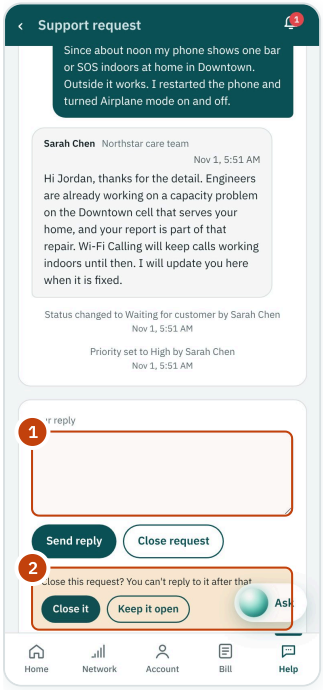


Figure 32. Replying and closing.

1. Reply box
2. Close asks for confirmation

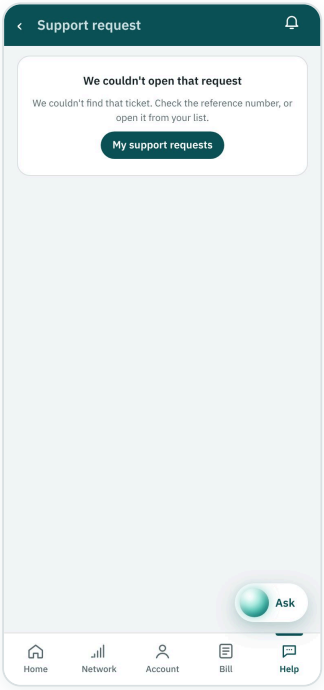


Figure 33. Opening a reference that is not yours or does not exist.

E4 Other screen sizes

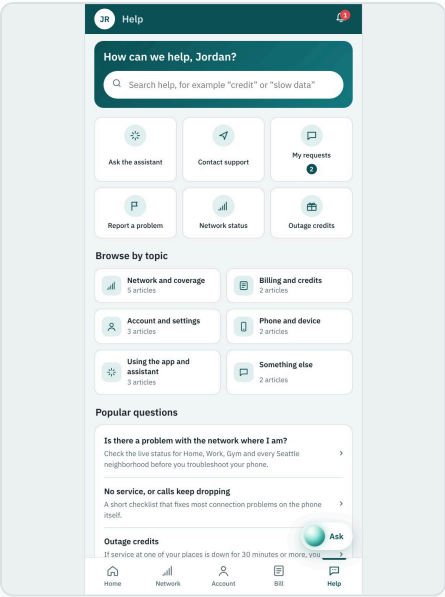


Figure 34. Help center on a tablet, 768 pixels wide. The phone-width column stays centred; no horizontal scrolling.

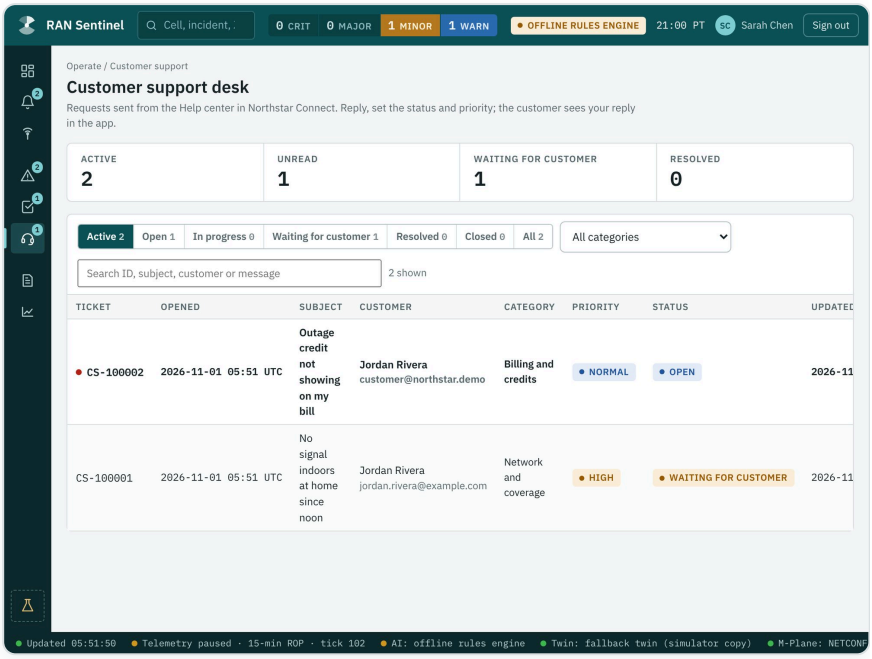


Figure 35. Support desk at 1024 by 768 pixels. Columns wrap and the table scrolls inside its panel when needed; the page itself does not scroll sideways.

SECTION F

F. Testing guide

The project has two kinds of automated test: a Python suite that exercises the backend through its real HTTP routes, and browser checks that drive the real pages in Chromium. Both were run for this edition, and the results below are copied from those runs.

F1 How to run the tests

Run in: the project folder, with the virtual environment active

```
python -m pytest -q                       # everything, about 5 minutes
python -m pytest -q tests/test_support.py  # the support module only, about 70 seconds
```

The Python tests need no API key and no running server: they build an offline instance of the system with an in-memory database. The browser checks need the server running and Node.js with Playwright (Section C8).

Run in: a second terminal in the project folder, while py run.py is running

```
node scripts/check_support_ui.js     # Help &amp;amp; Support on phone, tablet, desktop + support desk
node scripts/check_live_refresh.js   # live refresh of both apps
```

To point the browser checks at another address, set `BASE_URL`, for example `BASE_URL=http://127.0.0.1:8013`. They read the demo logins from `.env` or use the defaults. Note that the support check creates three real requests in your database; delete `data/sentinel.db` afterwards if you want an empty desk.

F2 Results of this run

Suite	What it covers	Tests	Result
Full Python suite	Safety rules, M-Plane rollback, detection, five fault scenarios, Muse Spark code path with a scripted API, care layer, maps, verifier, resume, voice agent, security, support module	177	**177 passed**
`tests/test_support.py`	The new module: validation, creation, persistence, access control, status changes, history, admin actions, help search (F3)	42	**42 passed**
`scripts/check_support_ui.js`	Help & Support in a real browser at three sizes, and the support desk (F4)	53 checks	**53 of 53 passed**
`tests/test_security.py`	Portal separation, lockout, file upload hardening	6	**Included above**

Environment: Linux container, Python 3, Chromium from Playwright, offline rules engine, run on 1 November 2026. The suite was earlier run on Windows by the author; this edition's numbers come from the container run.

F3 Support module test cases (executed)

Each row is an automated test in `tests/test_support.py`. "Actual" is what the test observed; every row passed.

ID	Name	Objective	Preconditions	Steps	Data	Expected	Actual	Status
TC-01	**Create a request**	A valid form creates a ticket	Signed in as customer	POST the form	Valid name, email, network category, subject, description	200, reference CS-100001, status Open, first message stored, owner hidden	As expected	**Pass**
TC-02	**Unique sequential references**	IDs never repeat	Signed in	Create five requests	Five subjects	CS-100001 to CS-100005	As expected	**Pass**
TC-03	**Field validation (13 cases)**	Bad input is refused with a field message	Signed in	POST with one bad field	Empty, too short, too long, bad email, script in email, unknown category	422 with the right message per field; nothing stored	As expected, 13 of 13	**Pass**
TC-04	**Malformed requests**	Wrong types and broken JSON are refused	Signed in	POST a list as subject, non-JSON, empty object	As listed	422	As expected	**Pass**
TC-05	**Input cleaning**	Control characters removed, HTML kept as text	Signed in	POST text with a bell character, tabs, extra blank lines and an img tag	As listed	Cleaned text stored verbatim; UI escapes it	As expected	**Pass**
TC-06	**Persistence across restart**	Tickets survive a new database connection	SQLite file in a temp folder	Create, reply, reopen the file twice	One ticket	Ticket, status and three messages read back	As expected	**Pass**
TC-07	**Survives demo reset**	Simulation reset keeps support data	SQLite file	Create, call reset	One ticket	Still one ticket	As expected	**Pass**
TC-08	**Own tickets only**	A customer cannot see another's ticket	Ticket owned by another email	List, open, reply, close it	Other customer's reference	Not listed; 404 on open, reply and close; nothing written	As expected	**Pass**
TC-09	**Second customer session**	Isolation between two signed-in customers	Two sessions	Second session lists and opens the first's ticket	Reference of the first	Empty list; 404	As expected	**Pass**
TC-10	**Sign-in required (11 endpoints)**	No support endpoint works without a session	Signed out	Call each endpoint	None	401 on all 11	As expected, 11 of 11	**Pass**
TC-11	**Customer cannot act as staff**	Customer session refused on staff endpoints	Signed in as customer	List all, update status, staff reply	Own reference	401	As expected	**Pass**

ID	Name	Objective	Preconditions	Steps	Data	Expected	Actual	Status
TC-12	**Staff cannot use customer endpoints**	Separate portals	Signed in as engineer	List and create through customer routes	Valid form	401	As expected	**Pass**
TC-13	**Full conversation**	Reply, read flags, reopen, close, history	Both sessions	Create, staff reads and replies, customer reads, replies, staff resolves, customer replies, closes, staff reopens	Messages	Statuses Open, Waiting, Open, Resolved, Open, Closed, Open; unread counts move as expected; 409 on reply to closed	As expected	**Pass**
TC-14	**Staff filters and search**	Status, category and text search	Two tickets, one resolved	Filter and search	Billing category, a word from a message, the reference	Correct ticket each time; counts right; 400 on unknown filter	As expected	**Pass**
TC-15	**Staff input checks**	Bad status, priority and empty reply refused	One ticket	Send invalid updates and replies	deleted, p0, spaces, 4,001 characters	422 or 400; 404 for unknown ticket; nothing written	As expected	**Pass**
TC-16	**Customer reply checks**	Empty or long replies refused	One ticket	Reply with empty and 4,001 characters	As listed	422; 404 for unknown reference	As expected	**Pass**
TC-17	**Open request limit**	Stops flooding	Signed in	Create 21 requests	Same form	21st refused with 429 and a clear message	As expected	**Pass**
TC-18	**Help search and articles**	Search ranks correctly; unknown items handled	Signed in	Search 'credit', 'slow data', nonsense; filter billing; open articles	As listed	Best article first; empty list; 400 and 404 where expected	As expected	**Pass**
TC-19	**Article assets are real**	No broken image or link	Repository	Check every image file and in-app link	17 articles	All images exist; all links go to real pages	As expected	**Pass**

F4 Browser checks (executed)

`scripts/check_support_ui.js` ran 53 checks: fourteen for each of three viewports (phone 360 by 740, tablet 768 by 1024, desktop 1440 by 900) and eleven for the support desk and the live hand-over between the two apps.

ID	Check	What is verified	Result
UI-01	**Help center renders**	Topics and popular questions shown; no sideways scroll	**Pass at 360, 768 and 1440 px**
UI-02	**FAQ, topic filter and search**	Expands; filter shows only that topic; 'credit' finds Outage credits; nonsense shows the empty state	**Pass at all three sizes**
UI-03	**Article screenshot**	Image loads (natural width above 0)	**Pass at all three sizes**
UI-04	**Form validation**	Four invalid fields marked with aria-invalid and messages	**Pass at all three sizes**
UI-05	**Accessible names**	Every visible button, link and field has a label or text	**Pass at all three sizes**
UI-06	**Create, list, reply**	Reference shown; request listed; reply added	**Pass at all three sizes**
UI-07	**Support desk**	New request listed; search narrows to one; draft kept through refresh; reply and priority saved; fits at 1024 and 1440 px	**Pass**
UI-08	**Live update to customer**	Reply and new status appear without reloading; Help tab dot appears and clears after reading	**Pass**
UI-09	**No script errors**	No JavaScript errors in either app during the run	**Pass**

F5 Recommended tests not yet executed

These were not run for this edition. They are listed so that nobody mistakes them for verified behaviour.

Test	Why it matters	How to run it
Screen reader walkthrough	The automated check confirms labels and invalid states, not the spoken experience.	NVDA or VoiceOver through Help, the form and a ticket.

Test	Why it matters	How to run it
Safari and Firefox	Checks ran in Chromium only.	Repeat Section G in each browser.
Real phone	Phone layout was checked by viewport size, not on a device.	Open the app over the local network on a phone.
Load and concurrency	SQLite with one lock is fine for a demo; many simultaneous writers were not measured.	A load tool such as Locust against the create and reply routes.
Live Meta models	This run used the offline engine. The support module uses no AI, but the assistant and agents do.	`py scripts\check_meta.py` and `py scripts\audit_agents.py` with a key.
Email delivery	The reply email is stored and shown to staff; no email is sent, by design.	Out of scope for the demo.

SECTION G

G. Project demonstration guide for judges

This script shows the whole product in about eight minutes on one computer. A shorter five-minute route is marked. It needs no API key; with a Meta key the same steps show the Meta models at work.

G1 Before you start (2 minutes, off the clock)

Start the app (Section C3 or C4) and open two browser windows side by side: the operations console at `/ops/` signed in as the engineer, and the customer app at `/connect/` signed in as the customer. Narrow the customer window to phone width. If you want an empty support desk, delete `data/sentinel.db` before starting. Open Demo controls once so you know where it is.

G2 The eight-minute route

Time	Where	Do this	Point out
0:00	Ops	Show the Network overview. Open Demo controls, choose SEA-RAN-001, *Capacity surge*, severity 2, click **Create problem**, then **Advance 15 min** twice.	Detection is a classic ML detector, not a language model; an incident opens on its own.
1:00	Ops	Open the new incident. Walk the workflow from Detector to Department. Open Diagnosis, then Plans and safety.	Verified diagnosis with evidence for and against; nine safety rules and the twin checked the plan before a person saw it.
2:30	Ops	In the decision card, approve the change.	M-Plane change package with automatic rollback; the cell is watched until it recovers.
3:15	Connect	Show Home: the problem card, the repair stage and the recovery estimate.	The customer hears first, in plain words, from facts engineers approved.
4:00	Connect	Open Help. Type “credit” in the search, open the article, then go back and tap **Contact support**. Submit once with the subject empty to show the error, then fill it in and send.	Searchable help with real screenshots; clear validation; reference number on success.
5:15	Ops	Open Customer support. The new request is bold with a red dot. Open it, type a reply, leave the status as Waiting for customer, click **Send reply**, set priority High and save.	Filters, search and the customer's other requests; the draft survives the live refresh.
6:15	Connect	Without reloading, the Help tab shows a red dot and a *The care team replied* card. Open the request, reply, then close it.	Two apps, one record; status history in the conversation; customers only ever see their own requests.

Time	Where	Do this	Point out
7:15	Ops	Optional: switch on *Diagnostics outage*, create an External interference fault and show the field-evidence request; attach a file from `samples/field_evidence/`.	The agent asks a person when data cannot settle the cause.
8:00	Ops	Finish on the Safety ledger.	Every detection, model call, check, decision and change is recorded.

Five-minute route: do the rows at 0:00, 1:00, 4:00, 5:15 and 6:15.

G3 If something goes wrong during the demo

If no incident appears after a minute, click **Advance 15 min** again; detection waits for two telemetry windows. If the map is blank, the street tiles need internet, and the state outlines and markers still work. If a page looks stale, the status bar says when it last refreshed; it retries by itself. **Reset demo** restarts the simulated network without deleting support requests. If the customer window shows the sign-in page, the server was restarted: sessions are kept in memory, so sign in again.

SECTION H

H. Troubleshooting and FAQ

H1 Common problems

Problem	Likely cause	Fix
`py` or `python` is not recognised	Python not installed or not on PATH	Install Python 3.10+ from python.org with "Add to PATH" ticked, then open a new terminal.
`activate` cannot be loaded in PowerShell	Script execution policy	Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once.
`pip install` fails	Old pip, no internet, or the venv not active	Activate the venv, run `python -m pip install --upgrade pip`, retry.
Address already in use, port 8000	Another copy of the server is running	Close the other window, or stop the process using port 8000.
"Email or password is incorrect"	Values in `.env` differ from what you typed, or the server was not restarted after editing	Check `OPS_*` and `CUSTOMER_*` in `.env`; restart.
"Too many failed sign-ins"	Five failures in five minutes	Wait the time shown, or restart the server.
Sent back to the sign-in page	Server restarted; sessions are in memory	Sign in again.
"Offline rules engine" everywhere	`MODEL_API_KEY` empty, or `.env` not in the project folder	Add the key to `.env` in the project folder and restart; check with `scripts\check_meta.py`.
Map shows only outlines	No internet for the street tiles	Expected offline; markers and coverage still work.
Contact form will not send	A field is invalid	Fix the fields outlined in red; each says what is wrong.
"You already have 20 open requests"	The per-customer limit	Close requests you no longer need, or reply on an existing one.
Old support requests after a reset	Support data is kept on purpose	Stop the server and delete `data/sentinel.db`.

Problem	Likely cause	Fix
Browser check: "Playwright is not installed"	Node packages missing	`npm i --no-save playwright` and `npx playwright install chromium`.
Maveric not answering	Docker stack not running, or Windows line endings broke the Kafka script	See `docs/MAVERIC_TWIN.md`; the app works without Maveric.

H2 Frequently asked questions

Does the support module use AI? No. Help articles are written content and search is a fixed ranking. Requests are answered by a person in the console. The AI assistant is a separate page.

Are emails sent? No. The reply email is stored with the request and shown to the care team, but the demo sends no email, and the app says replies appear in the app.

Can one customer see another's requests? No. The server links each request to the signed-in account and refuses any other; tests TC-08 and TC-09 check this.

Where is the data stored? In `data/sentinel.db` in the project folder. It is not committed to Git.

Why is there only one customer? The operator and subscriber are fictional demo data. The support code works per account, so adding a real identity provider would give each customer their own requests without code changes in `support.py`.

Is it ready for production? Not as is. Production would need real identity and roles instead of the two demo logins, persistent sessions, HTTPS, email or push notifications, and a server database for many concurrent users. The access rules, validation and history are written so that they carry over.

Are the help screenshots real? Yes. Every image in an article and in this manual was captured from the running app; none are mock-ups or stock images.

APPENDIX

Appendix

X1 Folder structure

```
Meta-Ai-Global-Hackathon-Ran-Sentinel-/
├── run.py                  start the server (port 8000)
├── start.bat               one-click start for Windows
├── requirements.txt        Python packages
├── .env.example            settings template (copy to .env, never commit .env)
├── backend/app/
│   ├── main.py             routes, sign-in, sessions
│   ├── orchestrator.py     incident workflow
│   ├── agents.py verify.py safety.py twin.py mplane.py
│   ├── simulator.py detection.py radio.py
│   ├── care.py             customer care layer
│   ├── support.py          Help & Support module
│   └── store.py            SQLite access
├── frontend/
│   ├── ops/                operations console
│   ├── connect/            Northstar Connect
│   │   └── help/           screenshots used in help articles
```

```
│   └── shared/             shared JS, CSS, map libraries
├── tests/                  pytest suite (177 tests)
├── scripts/                checks and tools (check_support_ui.js, check_meta.py, …)
├── samples/field_evidence/ sample photos and files for field evidence
├── docs/                   documentation and PDFs
└── data/                   created on first run: sentinel.db (not in Git)
```

X2 Support data limits

Field	Rule
Name	2 to 80 characters
Email	Up to 254 characters, form name@domain.tld
Subject	4 to 120 characters, single line
Category	network, billing, account, device, app or other
Description and replies	10 to 4,000 characters (replies: 1 to 4,000); control characters removed; line breaks kept
Open requests	At most 20 active per customer
Status	open, in_progress, waiting_customer, resolved, closed
Priority	low, normal (default), high, urgent

X3 Glossary

Term	Meaning
NOC	Network operations centre, the team that watches the network.
Cell, site, sector	A site is a tower location; each has three sectors, each sector is a cell.
KPI	Key performance indicator, such as drop rate or throughput.
PRB utilisation	How much of a cell's radio capacity is in use.
M-Plane	The O-RAN management interface used to change a radio's configuration.
Digital twin	A model of the network used to test a change before applying it.
Ticket (department)	A decision sent to an engineering team, IDs TCK-0001 and up.
Support request	A customer's message to the care team, IDs CS-100001 and up.
Offline rules engine	The fallback that replaces the Meta models when no key is set.

X4 Related documents

`README.md` (overview), `DEMO.md` (three-minute video script), `docs/SETUP_AND_DEPLOYMENT.md`, `docs/API_AND_DATABASE.md`, `docs/SECURITY_AND_ACCESS_CONTROL.md`, `docs/RAN_Sentinel_Technical_Documentation.pdf`.